

Diplomarbeit

EduShelf

Verfasser: Elias Welt 5AHITS
Phillip Rubak 5AHITS

Betreuer: Dipl.-Ing. Gerald Stoll

Schuljahr: 2025/26

Abgabevermerk:

Datum: 07.04.2025

übernommen von:

Höhere Technische Bundeslehranstalt Hollabrunn

Höhere Lehranstalt für Informationstechnologie

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche erkenntlich gemacht habe.

Hollabrunn, am 07.04.2025, Elias Welt

Hollabrunn, am 07.04.2025, Phillip Rubak

1 HINWEISE

Die vorliegende Diplomarbeit wurde in Zusammenarbeit mit der Firma "X-Works" ausgeführt.

Die in dieser Diplomarbeit entwickelten Prototypen und Software-Produkte dürfen ganz oder auch in Teilen von Privatpersonen oder Firmen nur dann in Verkehr gebracht werden, wenn sie diese selbst geprüft und für den vorgesehenen Verwendungszweck für geeignet befunden haben.

Es wird keinerlei Haftung übernommen für irgendwelche Schäden, die aus der Nutzung der hier entwickelten oder beschriebenen Bestandteile des Projekts resultieren.

Für alle Entwicklungen gilt die GNU General Public License [<http://www.gnu.org/licenses/gpl.html>] der Free Software Foundation, Boston, USA in der Version 3.

2 SCHLÜSSELBEGRIFFE

- EduShelf
- Backend
- Frontend
- Retrieval-Augmented Generation (RAG)
- Large Language Model (LLM)

3 DANKSAGUNGEN

Wir möchten an dieser Stelle unseren aufrichtigen Dank all jenen aussprechen, die uns während der Entstehung dieser Diplomarbeit unterstützt haben.

Unser Dank gilt XWorks, unserer Partnerfirma, die uns am start mit wertvollen Einblicken und praxisnaher Unterstützung begleitet haben und ebenfalls bei der Ideen und Entscheidungsfindung geholfen haben. Die Zusammenarbeit mit euch war eine ausschlaggebende Hilfe für das Projekt.

Ebenso möchten wir uns herzlich bei Herrn Stoll, unserem Diplomarbetsbetreuer, bedanken. Durch sein fundiertes Fachwissen, seine wertvollen Impulse und sein kontinuierliches Engagement hat er die Qualität unserer Ergebnisse entscheidend geprägt

Ein herzlicher Dank gilt allen, die uns auf diesem Weg begleitet haben. Ob durch konstruktives Feedback, motivierenden Zuspruch oder die nötige Geduld – diese vielfältige Unterstützung war essenziell für das Gelingen unserer Diplomarbeit.

DIPLOMARBEIT DOKUMENTATION

Name der Verfasser/innen	Elias Welt, Phillip Rubak
Jahrgang Schuljahr	2025/26
Thema der Diplomarbeit	EduShelf
Kooperationspartner	—

Aufgabenstellung	Im Rahmen dieser Diplomarbeit wird 'EduShelf' entwickelt, eine webbasierte Plattform zur Verwaltung und Interaktion mit Bildungsunterlagen. Ziel ist es, Schülern und Lehrern eine zentrale Lösung zu bieten, um Dokumente zu organisieren und KI für eine effiziente Informationsbeschaffung zu nutzen. Die Anwendung verbindet Dokumentenmanagement mit einem KI-gestützten Chat-System, das es Benutzern ermöglicht, Fragen zu ihren hochgeladenen Inhalten zu stellen (RAG - Retrieval Augmented Generation). Ein zentraler Aspekt ist die nahtlose Integration eines modernen Web-Frontends mit einem leistungsfähigen Backend und einem KI-Server. Das Backend steuert die Datenverarbeitung und -speicherung, während der KI-Server die Intelligenz für die Chat-Funktionalität bereitstellt. Diese Plattform soll den Zugriff auf und die Nutzung von Bildungsressourcen modernisieren.
------------------	--

Realisierung	Das Projekt umfasst die Entwicklung einer responsiven Webanwendung und eines leistungsfähigen Backend-Systems. Das Frontend wird mit React umgesetzt und bietet eine moderne Benutzeroberfläche. Das Backend, implementiert in .NET, steuert die Datenspeicherung und -verarbeitung. Ein dedizierter KI-Server, basierend auf Python und Ollama, übernimmt die LLM-Operationen für die Chat-Funktionalität. Das System nutzt Containerisierung (Docker) für einfache Bereitstellung und Skalierbarkeit.
--------------	---

Ergebnisse	Das Ergebnis ist die voll funktionsfähige Plattform 'EduShelf'. Zu den Kernfunktionen gehören Benutzerauthentifizierung, Upload und Indizierung von Dokumenten sowie ein interaktiver KI-Chat. Nutzer können Accounts erstellen, ihre Dokumentenbibliothek verwalten und durch die KI-Schnittstelle sofortige Antworten aus ihren Lernunterlagen erhalten. Das System demonstriert die effektive Integration moderner Webtechnologien und künstlicher Intelligenz im Bildungsbereich.
------------	---

Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbild.png
---	---

Teilnahme an Wettbewerben, Auszeichnungen	
--	--

Möglichkeiten der Einsichtnahme in die Arbeit	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn
--	--

Approbation (Datum / Unterschrift)	Prüfer/Prüferin	Direktor/Direktorin Abteilungsvorstand/ Abteilungsvorständin
---------------------------------------	-----------------	--

DIPLOMA THESIS

Documentation

Author(s)	Elias Welt, Phillip Rubak,
From Academic year	2025/26
Topic	EduShelf
Co-operation partners	TailoredApps

Task Description	This diploma project focuses on developing 'EduShelf', a web-based platform for managing and interacting with educational content. The goal is to provide students and teachers with a centralized solution for organizing documents and leveraging AI for efficient information retrieval. The app integrates document management with an AI-powered chat system that allows users to ask questions about their uploaded content (RAG - Retrieval Augmented Generation). A key aspect is the seamless integration of a modern web frontend with a powerful backend and an AI server. The backend handles data processing and storage, while the AI server provides the intelligence for the chat functionality. This platform aims to modernize how educational resources are accessed and utilized.
------------------	---

Implementation	The project involves developing a responsive web application and a robust backend system. The frontend is built with React, ensuring a modern and user-friendly interface. The backend, implemented in .NET, manages data storage and processing. A dedicated AI server, utilizing Python and Ollama, handles the large language model operations for the chat functionality. The system uses containerization (Docker) for easy deployment and scalability.
----------------	---

Results	The expected outcome is a fully functional 'EduShelf' platform. Key features include user authentication, document upload and indexing, and an interactive AI chat. Users can create accounts, manage their document libraries, and obtain instant answers from their study materials through the AI interface. The system demonstrates the effective integration of modern web technologies and artificial intelligence in an educational context.
---------	---

Typische Grafik, Foto etc. (mit Erläuterung)	MISSING SVG: img/blockschaltbild.png
---	--------------------------------------

Participation in competitions Awards	
---	--

Accessibility of final project thesis	HTL Hollabrunn Anton Ehrenfriedstraße 10 2020 Hollabrunn
--	--

Approval (Date / Signature)	Examiner/s	Head of Department / College
-----------------------------	------------	------------------------------

Thema

EduShelf

Verlauf

- 21.05.2025: Das Thema 'xWorks KI' wurde durch Gerald Stoll angelegt.
- 09.09.2025: Sie wurden zu Thema hinzugefügt
- 09.09.2025: Sie wurden zu Thema hinzugefügt
- 15.09.2025: Thema wurde eingereicht
- 15.09.2025: Thema wurde von Gerald Stoll akzeptiert
- 15.09.2025: Thema wurde von Wilfried Trollmann akzeptiert
- 29.09.2025: Thema wurde von Wolfgang Bodei akzeptiert

Schulische Informationen

- **Schulart:** Technische, gewerbliche und kunstgewerbliche Schulen
- **Klasse:** 5AHITS
- **Abteilung:** IT
- **Schuljahr der abschließenden Prüfung:** 2025/26

Betreuung

Rolle	Name
Direktion	Wolfgang Bodei (Akzeptiert)
Abteilungsvorstand	Wilfried Trollmann (Akzeptiert)
Hauptverantwortlicher Betreuer	Gerald Stoll (Akzeptiert)

Kooperationspartner / Auftraggeber

- **Name:** X-WORKS software GmbH
- **Adresse:** Mankerstraße 24, 3380 Pöchlarn
- **URL:** <https://x-works.at/>
- **E-Mail:** info@x-works.at
- **Ansprechpartner:** Georg Ungerböck
- **Telefon:** +43 2757 24131
- **Typ:** Sonstige

Beteiligte SchülerInnen

Schüler/in	Individuelle Themenstellung	Abteilu
Elias Welt	Frontend, Frontend/Backend Schnittstelle, KI Unterstützung	IT
Phillip Rubak	Backend, Datenbank, KI, RAG	IT

Ausgangslage

Lernmaterialien entwickeln sich oft sehr schnell zu einer riesigen und unübersichtlichen Dokumentenansammlung. Dies führt dazu, dass es für die Schüler oft sehr schwierig ist daraus die wichtigsten Inhalte zu extrahieren, was in dem sonst schon sehr stressigen Schulalltag zusätzlich nochmal sehr viel Zeit kostet.

Untersuchungsanliegen

Die Digitalisierung des Bildungswesens hat zu einer starken Zunahme digitaler Lernmaterialien geführt. Diese liegen häufig verteilt auf verschiedenen Plattformen, was eine strukturierte Verwaltung und gezielte Nutzung erschwert. Gleichzeitig fehlen Lösungen, die nicht nur der Organisation dienen, sondern Lernende aktiv beim Verstehen und Verarbeiten von Inhalten unterstützen.

Die Untersuchung orientiert sich an drei Leitfragen: Wie lässt sich eine plattformunabhängige Lösung gestalten, die den Bedürfnissen von Schüler:innen, Lehrkräften und Studierenden gerecht wird? In welchem Umfang können KI-Funktionen die Arbeit mit Lernmaterialien verbessern? Welche Anforderungen an Datenschutz, Sicherheit und Barrierefreiheit sind für tatsächlichen schulischen Einsatz entscheidend?

Diese Arbeit fokussiert sich auf eine Website mit den Kernmodulen Benutzerverwaltung, Dateiverwaltung, Kategorisierung, Suche und ausgewählten KI-Funktionen. Erweiterungen wie eine mobile App oder Gamification werden als mögliche Zusatzfeatures betrachtet.

Zielsetzung

Entwicklung einer webbasierten Plattform zur Verwaltung, Kategorisierung und Nutzung von schulischen Lernmaterialien (PDF, Markdown, Textdateien). Nutzer sollen Materialien hochladen, mit Tags versehen, durchsuchen und mit einer KI interagieren können, um Inhalte besser zu verstehen oder zusammenfassen zu lassen.

Geplantes Ergebnis

Im Rahmen der Diplomarbeit entsteht eine webbasierte Plattform zur zentralen Verwaltung und Nutzung schulischer Lernmaterialien. NutzerInnen können Dateien wie PDF, Markdown oder Text hochladen, mit Tags versehen, durchsuchen und organisieren. Ein responsive Frontend sorgt für einfache Bedienung auf Desktop sowie mobilen Endgeräten.

Als Herausstellungsmerkmal gegenüber bereits vorhandenen Verwaltungsmöglichkeiten werden KI-gestützte Funktionen integriert (automatische Zusammenfassungen, Quiz- und Lernkartengenerierung sowie vereinfachte Erklärungen). Ermöglicht wird auch die direkte Interaktion mit Dokumenten via AI-Chatbot. Das geplante Ergebnis umfasst eine funktionsfähige Plattform mit Benutzerverwaltung, Datei-Upload, Suche, Tagging und ausgewählten KI-Funktionen sowie eine Dokumentation aller unserer Schritte. Ziel ist ein Tool, welches für Effizienz und Zeitersparnis steht.

Meilensteine

Meilenstein	Datum
Start Diplomarbeit	10.04.2025
Prototyp fertig	01.01.2026
Tests beenden	01.02.2026
Dokumentation	15.03.2026
Abgabe fertige Diplomarbeit	27.03.2026

Erklärung

Die unterfertigten Kandidaten/Kandidatinnen haben gemäß den geltenden schulrechtlichen Bestimmungen die Ausarbeitung einer abschließenden Arbeit (Diplomarbeit bzw. Abschlussarbeit) mit folgender Aufgabenstellung gewählt:

EduShelf - KI basierte Lernplattform

Individuelle Aufgabenstellungen im Rahmen des Gesamtprojektes:

- Elias Welt (5AHITS): **Frontend, Frontend/Backend Schnittstelle, AI, RAG (Retrieval-Augmented Generation)**
- Phillip Rubak (5AHITS): **Backend, Datenbank, AI**

Die Kandidaten/Kandidatinnen nehmen zur Kenntnis, dass die abschließende Arbeit in eigenständiger Weise und außerhalb des Unterrichtes zu bearbeiten und anzufertigen ist, wobei Ergebnisse des Unterrichtes mit einbezogen werden können, die jedenfalls als solche entsprechend kenntlich zu machen sind.

Die Abgabe der vollständigen abschließenden Arbeit hat in digitaler und in zweifach ausgedruckter Form bis spätestens **27.03.2026** beim zuständigen Betreuer/der zuständigen Betreuerin zu erfolgen.

Die Kandidaten/Kandidatinnen nehmen auch zur Kenntnis, dass ein Abbruch der abschließenden Arbeit nicht möglich ist.

Kandidaten/Kandidatinnen:

**Datum und Unterschrift bzw.
Handysignatur:**

Elias Welt (5AHITS)

Phillip Rubak (5AHITS)

Inhaltsverzeichnis

1	HINWEISE	5
2	SCHLÜSSELBEGRIFFE	6
3	DANKSAGUNGEN	7
4	Einführung	23
4.1	Motivation	23
4.2	Umfeld und Stand der Technik	23
4.2.1	Existierende Lösungen und ihre Grenzen	23
4.2.2	Retrieval-Augmented Generation (RAG)	24
4.3	Aufgabenstellung	24
4.3.1	Funktionale Anforderungen	25
4.3.2	Nicht-funktionale Anforderungen	25
4.4	Aufbau der Arbeit	26
4.5	Entstehung der Idee	27
4.5.1	Stufe 1 – Problemidentifikation: Unkontrollierter Datei- anstieg	27
4.5.2	Stufe 2 – Lösungsansatz: Zentralisierter Speicher- und Sortierort	27
4.5.3	Stufe 3 – Erweiterung: KI-gestützte Lernunterstützung	28
4.5.4	Stufe 4 – Synthese: EduShelf als Gesamtprodukt	28
5	Frontend Implementierung	29
5.1	Einführung und Zielsetzung	29
5.2	Das Dashboard – Zentrale Übersichtsansicht	30
5.2.1	Architektur und Datenladen	30

5.2.2	Filterung geteilter Dateien	31
5.2.3	Schnellaktionen und Statistik-Widget	31
5.3	Authentifizierungsflow: Registrierung und E-Mail-Bestätigung	32
5.3.1	Registrierung (Register.jsx)	33
5.3.1.1	Clientseitige Validierung	33
5.3.1.2	Serverseitige Fehlerverarbeitung	34
5.3.2	Login-Prozess (Login.jsx)	34
5.3.3	E-Mail-Bestätigung	35
5.4	Navigationsstruktur und Layout (MainLayout.jsx)	35
5.4.1	Outlet-Pattern und verschachtelte Routen	36
5.4.2	Rollenbasierte Navigation (RBAC)	36
5.4.3	Aktiver Zustand mit NavLink	37
5.5	Datei-Sharing zwischen Nutzern	37
5.5.1	Share-Dialog (ShareFileModal.jsx)	37
5.5.2	Zustandsverwaltung für Shared Files	39
5.5.3	Bewertung des Ansatzes	39
5.6	Technologie-Stack	40
5.6.1	React (v19)	40
5.6.2	Vite	40
5.6.3	React Router (v7)	40
5.7	Projektsetup und Konfiguration	41
5.8	Softwarearchitektur und Datenfluss	42
5.8.1	Unidirektionaler Datenfluss	42
5.8.2	Routing-Strategie	42
5.9	Datenschicht und API-Integration	43
5.9.1	API-Wrapper	43
5.10	Implementierung der Kernkomponenten	44
5.10.1	Authentifizierungssystem	44
5.10.2	Dateimanagement (Files.jsx)	44
5.10.3	Interaktives Lernen: Das Quiz-Modul	45
5.10.3.1	Datenstruktur	45
5.10.3.2	Ablauf eines Quiz	45
5.10.4	Flashcards (Lernkarten)	45

5.10.5	Chat und KI-Integration	46
5.10.6	Admin Panel und RBAC	46
5.11	Benutzeroberfläche und Design (UI/UX)	47
5.11.1	Styling-Strategie	47
5.11.2	Dark Mode Implementierung	47
5.12	Detaillierte Analyse ausgewählter Module	48
5.12.1	Quiz-Editor Logik (QuizModal.jsx)	48
5.12.1.1	Zustandsverwaltung	48
5.12.1.2	Dynamische Formular-Manipulation	48
5.12.1.3	Validierung und Speichern	49
5.12.2	Flashcards: 3D-Interaktion und State	49
5.12.3	Admin Panel: Kaskadierende Datenabfragen	49
5.13	Deployment und Betrieb	50
5.13.1	Docker-Containerisierung	50
5.13.2	Nginx Konfiguration	51
5.14	Herausforderungen und Lösungen	51
5.14.1	Synchronisation des UI-Status	51
5.14.2	Performance bei großen Dateilisten	52
5.15	Ausblick und Erweiterungsmöglichkeiten	52
5.16	Fazit	52
6	Backend Implementierung	55
6.1	Systemarchitektur	55
6.1.1	Presentation Layer (Controllers)	55
6.1.2	Business Logic Layer (Services)	56
6.1.3	Data Access Layer (Data)	56
6.1.4	Infrastructure Layer	57
6.2	Technologie-Stack	57
6.3	Projekt-Setup und Konfiguration	58
6.3.1	Programm-Initialisierung (Program.cs)	58
6.3.2	Konfiguration (appsettings.json)	59
6.4	Datenmodelle und Datenbankdesign	60
6.4.1	Kernelemente	60

6.4.2	Entity Relationship Diagram (ERD)	62
6.5	API Design und Controller	62
6.5.1	Ausgewählte API-Endpunkte	63
6.6	Sicherheitskonzepte	63
6.6.1	Authentifizierung & Autorisierung	64
6.6.2	Rollenbasiertes Zugriffssystem (RBAC)	65
6.6.3	E-Mail-Verifizierung	65
6.6.3.1	Registrierungsvorgang	65
6.6.3.2	Bestätigungsvorgang	66
6.6.4	Datenvalidierung (FluentValidation)	66
6.6.5	Error Handling Middleware	67
6.7	Ingestion-Pipeline (Datenverarbeitung)	67
6.7.1	1. Textextraktion (OCR & Parsing)	67
6.7.2	2. Chunking (Intelligente Segmentierung)	68
6.7.3	3. Embedding und Vektorspeicherung	68
6.8	Detaillierte Analyse ausgewählter Module	69
6.8.1	Authentifizierung: Secure Cookies und Hashing	69
6.8.1.1	Passwort-Hashing (BCrypt)	69
6.8.1.2	Cookie-Konstruktion	70
6.8.2	Ingestion-Pipeline: HNSW Indexierung	70
6.9	RAG (Retrieval-Augmented Generation) Workflow	70
6.9.1	Systemübersicht und Datenfluss	71
6.9.2	Schritt 1: Intent-Erkennung (IntentDetectionService)	72
6.9.3	Schritt 2: Semantische Suche (RetrievalService)	72
6.9.3.1	Primäre Suche: Name-basiertes Matching	73
6.9.3.2	Fallback: Semantische Vektorsuche	73
6.9.4	Schritt 3: Prompt-Aufbau (PromptGenerationService)	74
6.9.5	Schritt 4: Orchestrierung und Intent-basiertes Branching (ChatService)	74
6.9.6	Zusammenfassung: Retrieval-Augmented Generation	75
6.10	KI-gestützte Lerninhaltsgenerierung	76
6.10.1	Generierungspipeline (5 Schritte)	76
6.10.2	Flashcard-Generierung (FlashcardService)	77

6.10.3 Quiz-Generierung (QuizService)	77
6.11 Infrastruktur & Deployment	78
6.11.1 Docker-Komposition	78
6.11.1.1 Datenbank (PostgreSQL mit pgvector)	78
6.11.1.2 KI-Inferenz (Ollama)	79
6.11.1.3 Object Storage (MinIO)	79
6.11.2 Dockerfile	80
6.12 Resilienz und Stabilität	80
6.12.1 Retry-Policies & Circuit Breaker	80
6.13 Herausforderungen und Lösungen	81
6.13.1 Blockierende Operationen bei großen Dateien	81
6.13.2 LLM Kontext-Fenster-Limits	82
6.13.3 Datenkonsistenz bei Löschoperationen	82
6.14 Qualitätssicherung und Tests	82
6.14.1 Unit Tests (xUnit + Moq)	83
6.14.1.1 Beispiel: FlashcardService	83
6.14.1.2 Beispiel: AuthService	84
6.14.2 Validatoren-Tests	85
6.14.3 Integration Tests	86
6.15 Fazit	86
6.15.1 Erreichte Ziele	86
6.15.2 Lernerkenntnisse und Herausforderungen	87
6.15.3 Mögliche Erweiterungen	87

4 Einführung

4.1 Motivation

Der Umgang mit Lernmaterialien im Bildungsalltag ist trotz fortschreitender Digitalisierung nach wie vor fragmentiert: Dokumente liegen in verschiedenen Cloud-Diensten verstreut, Zusammenfassungen werden manuell erstellt und das gezielte Heraussuchen von Informationen aus umfangreichen Skripten oder Büchern kostet wertvolle Lernzeit. Gleichzeitig haben Sprachmodelle (Large Language Models, LLMs) in den letzten Jahren einen Reifegrad erreicht, der eine sinnvolle Integration in produktive Anwendungen ermöglicht – ohne dabei auf kommerzielle Drittanbieter-APIs angewiesen zu sein, die den Datenschutz gefährden könnten.

EduShelf adressiert genau diese Lücke: Die Plattform kombiniert klassisches Dokumentenmanagement mit KI-gestütztem Lernen in einer einzigen, datenschutzfreundlichen Webanwendung. Nutzerinnen und Nutzer können ihre Unterlagen hochladen und anschließend direkt mit ihrem eigenen Lernmaterial interagieren – durch gezielte Fragen, automatische Lernkarten oder Multiple-Choice-Quizze, die das System aus dem Dokumenteninhalt generiert.

4.2 Umfeld und Stand der Technik

4.2.1 Existierende Lösungen und ihre Grenzen

Auf dem Markt existieren bereits Einzellösungen für Teilaspekte des Problems:

- **Cloud-Speicherdienste** (z.B. Google Drive, OneDrive) ermöglichen das Ablegen und Teilen von Dokumenten, bieten aber keine inhaltliche Interaktion mit den Dateien.
- **KI-Chatbots** (z.B. ChatGPT, Google Gemini) können Fragen beantworten, haben jedoch keinen Zugriff auf eigene, private Dokumente – oder

senden diese zur Verarbeitung an externe Server, was datenschutzrechtlich problematisch ist.

- **Lernkarten-Apps** (z.B. Anki) müssen manuell befüllt werden und sind nicht an eine Dokumentbibliothek angebunden.

Eine integrierte Plattform, die Dokumentenverwaltung, semantische Suche und KI-Lerninhalte in einem lokalen, datenschutzkonformen System vereint, existierte bisher nicht als Open-Source-Lösung für den Bildungsbereich.

4.2.2 Retrieval-Augmented Generation (RAG)

Der zentrale technische Ansatz von EduShelf ist *Retrieval-Augmented Generation* (RAG). Dabei wird ein LLM nicht mit seinem statischen Trainingswissen allein befragt, sondern mit relevantem Kontext aus einem externen Wissensspeicher angereichert. Konkret bedeutet das:

1. Hochgeladene Dokumente werden in kleine Textabschnitte (*Chunks*) zerlegt und als Vektoren in einer Datenbank gespeichert.
2. Bei einer Nutzeranfrage werden die semantisch passendsten Chunks per Vektorsuche abgerufen.
3. Diese Chunks werden als Kontext zusammen mit der Frage an das LLM übergeben.
4. Das LLM generiert eine präzise, auf das Dokument bezogene Antwort.

Dieses Verfahren reduziert Halluzinationen und ermöglicht es, ein lokal betriebenes Modell (ohne Internetzugang) mit dokumentenspezifischem Wissen zu versorgen.

4.3 Aufgabenstellung

Im Rahmen dieser Diplomarbeit an der HTL Hollabrunn wurde die Plattform **EduShelf** vollständig konzipiert, implementiert und dokumentiert. Das System besteht aus drei Hauptkomponenten, die von den beiden Diplomanden arbeitsteilig entwickelt wurden:

- **Web-Frontend** (Elias Welt): Eine React-basierte Single-Page-Application (SPA) mit vollständiger Benutzeroberfläche für Dokumentverwaltung, Chat, Lernkarten und Quizze.
- **Backend-API** (Phillip Rubak): Eine ASP.NET Core 8 Web API, die als zentrale Schnittstelle zwischen Frontend, Datenbank, KI-Modell und Dateispeicher fungiert.

4.3.1 Funktionale Anforderungen

Das System soll folgende Kernfunktionen bereitstellen:

1. **Benutzerverwaltung:** Registrierung mit E-Mail-Verifizierung, Login, rollenbasierte Zugriffskontrolle (Student / Admin).
2. **Dokumentenverwaltung:** Upload, Ansicht, Herunterladen und Löschen von Dokumenten verschiedener Formate (PDF, DOCX, TXT). Dokumente können mit anderen Nutzern geteilt werden.
3. **KI-Chatbot (RAG):** Nutzer können in natürlicher Sprache Fragen zu ihren Dokumenten stellen. Das System erkennt die Nutzerintention automatisch und ruft relevante Dokumentenabschnitte ab.
4. **Lernkarten-Generierung:** Aus Dokumenteninhalten werden automatisch Flashcards (Frage-Antwort-Paare) generiert, die in einem interaktiven Review-Modus geübt werden können.
5. **Quiz-Generierung:** Das System erstellt automatisch Multiple-Choice-Quizze aus hochgeladenen Dokumenten, inklusive Auswertung und Ergebnisanzeige.
6. **Tag-System:** Dokumente und Lernkarten können mit Tags versehen und gefiltert werden.

4.3.2 Nicht-funktionale Anforderungen

- **Datenschutz:** Alle KI-Operationen laufen lokal auf dem Server (via Ollama). Es werden keine Dokumenteninhalte an externe Dienste übertragen.
- **Deployment:** Das gesamte System ist containerisiert (Docker Compose) und auf einem einzelnen Linux-Server betreibbar.

- **Erweiterbarkeit:** Die Architektur ist modular aufgebaut, sodass neue KI-Modelle oder Features ohne Umbau des Gesamtsystems integriert werden können.
- **Sicherheit:** Passwörter werden mit BCrypt gehasht, Sessions über HttpOnly-Cookies verwaltet, und alle API-Endpunkte sind standardmäßig durch Authentifizierung geschützt.

4.4 Aufbau der Arbeit

Die vorliegende Diplomarbeit ist in zwei Hauptteile gegliedert:

- **Frontend-Dokumentation** beschreibt die React-Architektur, das Komponentendesign, das Routing, die API-Anbindung und das UX-Design der Webanwendung (Elias Welt).
- **Backend-Dokumentation** beschreibt die Systemarchitektur, das Datenmodell, die REST-API, die RAG-Pipeline, die KI-Dienste, die Infrastruktur sowie die Qualitätssicherung durch automatisierte Tests (Phillip Rubak).

Beide Kapitel folgen einer technikorientierten Struktur und belegen ihre Aussagen durch Codeauszüge, Diagramme und konkrete Implementierungsdetails.

4.5 Entstehung der Idee

Die konzeptionelle Grundlage von EduShelf resultiert aus einer systematischen Analyse des schulischen Arbeitsalltags. Im Zuge der Beobachtung eigener Lernprozesse sowie der Erfahrungen aus dem Schulbetrieb kristallisierten sich wiederkehrende Ineffizienzen heraus, die als Ausgangspunkt für die Produktentwicklung dienten. Nachfolgend wird der ideelle Entstehungsprozess anhand von vier aufeinander aufbauenden Erkenntnisstufen erläutert.



Abbildung 4.1: Konzeptionelle Entwicklung der Idee – von der Problemerkennung zur Lösung

4.5.1 Stufe 1 – Problemidentifikation: Unkontrollierter Dateianstieg

Im schulischen Kontext akkumuliert sich über die Jahre hinweg eine erhebliche Menge an digitalem Lernmaterial. Mitschriften, Übungsblätter, Präsentationen und Abgaben werden typischerweise in heterogenen Strukturen auf verschiedenen Endgeräten oder Cloud-Diensten abgelegt, ohne einer konsistenten Organisationslogik zu folgen. Diese Fragmentierung führt zu einem signifikanten Mehraufwand beim Wiederauffinden relevanter Unterlagen und beeinträchtigt die Effizienz des Lernprozesses nachhaltig. Die erste konzeptionelle Erkenntnis lautet daher: Die schiere Quantität an digitalem Lernmaterial stellt ohne geeignete Verwaltungsstruktur ein wesentliches Produktivitätshemmnis dar.

4.5.2 Stufe 2 – Lösungsansatz: Zentralisierter Speicher- und Sortierort

Aus der identifizierten Problematik ergibt sich die logische Konsequenz, einen einheitlichen, zentralisierten Ablageort für sämtliche Lernmaterialien zu schaf-

fen. Dieser sollte nicht lediglich als passives Dateisystem fungieren, sondern aktive Mechanismen zur strukturierten Klassifikation und Kategorisierung bereitstellen. Konkret bedeutet dies die Möglichkeit, Dokumente mit beschreibenden Metadaten (Schlagwörter, Fächerzuordnung) zu versehen und gezielt durchsuchbar zu machen. Die Anforderung an einen solchen Speicherort geht folglich über die Funktionalität konventioneller Cloud-Lösungen hinaus, indem eine lernerorientierte Organisationsstruktur in den Mittelpunkt gestellt wird.

4.5.3 Stufe 3 – Erweiterung: KI-gestützte Lernunterstützung

Die Integration von Methoden der Künstlichen Intelligenz stellt die entscheidende konzeptionelle Erweiterung gegenüber einer reinen Dokumentenverwaltungslösung dar. Die Fähigkeit, auf Basis des individuellen Dokumentenbestands automatisiert Lernhilfen zu generieren – sei es durch kontextbezogene Fragen und Antworten im Chat-Format, durch das automatische Erstellen von Übungsaufgaben oder durch die Zusammenfassung komplexer Inhalte – hebt das System auf eine qualitativ höhere Ebene. Dieser Schritt transformiert EduShelf von einem passiven Archiv zu einem aktiven Lernbegleiter, der das gespeicherte Wissen für den Anwender unmittelbar nutzbar macht.

4.5.4 Stufe 4 – Synthese: EduShelf als Gesamtprodukt

Die vorangehenden drei Erkenntnisstufen münden in der Produktvision von EduShelf. Die Bezeichnung verbindet die Kernfunktionen symbolisch: *Edu* (lat./engl. für Bildung) und *Shelf* (engl. für Regal) verweisen auf einen strukturierten, zugänglichen Wissensraum. EduShelf vereint die zentrale Dateiverwaltung mit intelligenten Lernfunktionen zu einer kohärenten Plattform, die Schülerinnen und Schülern ermöglicht, ihren gesamten Lernprozess – vom Ablegen der Unterlagen bis zur aktiven Wissensüberprüfung – in einer einzigen Anwendung zu bündeln. Abbildung 4.1 visualisiert diesen Gedankengang als nachvollziehbaren, vierstufigen Prozess von der Problemerkennung bis zur konkreten Produktidee.

5 Frontend Implementierung

5.1 Einführung und Zielsetzung

Das Frontend der Anwendung „EduShelf“ repräsentiert die Schnittstelle zwischen dem Benutzer und den Backend-Diensten. Ziel der Implementierung war die Schaffung einer modernen, performanten und intuitiven Benutzeroberfläche, die das Lernen und Verwalten von Dokumenten so effizient wie möglich gestaltet.

Im Gegensatz zu klassischen Webanwendungen, die auf dem „Multi-Page Application“ (MPA) Prinzip basieren und bei jeder Interaktion eine neue HTML-Seite vom Server anfordern, wurde EduShelf als „Single Page Application“ (SPA) konzipiert. In einer SPA wird lediglich beim ersten Aufruf eine einzige HTML-Datei (`index.html`) geladen. Alle weiteren Interaktionen finden clientseitig statt, indem JavaScript (in diesem Fall React) den DOM (Document Object Model) dynamisch manipuliert.

Die Vorteile dieses Architekturansatzes sind vielfältig:

- **Performance:** Da nach dem initialen Laden nur noch Rohdaten (im JSON-Format) und keine vollständigen HTML-Seiten mehr übertragen werden müssen, reduziert sich das Datenvolumen signifikant.
- **Benutzererfahrung (UX):** Übergänge zwischen verschiedenen Ansichten erfolgen nahezu latenzfrei, ohne das typische Flackern eines Seiten-Reloads. Dies vermittelt das Gefühl einer nativen Desktop-Anwendung.
- **Entkopplung:** Durch die strikte Trennung von Frontend und Backend über eine REST-API können beide Systeme unabhängig voneinander entwickelt, getestet und skaliert werden.

5.2 Das Dashboard – Zentrale Übersichtsansicht

Die Startseite der Anwendung, realisiert in der Komponente `MainDashboard.jsx`, stellt nach erfolgreicher Authentifizierung den primären Einstiegspunkt für den Nutzer dar. Sie verfolgt das Konzept eines persönlichen Lernportals, das auf einen Blick die relevantesten Informationen und die häufigsten Aktionen bündelt.

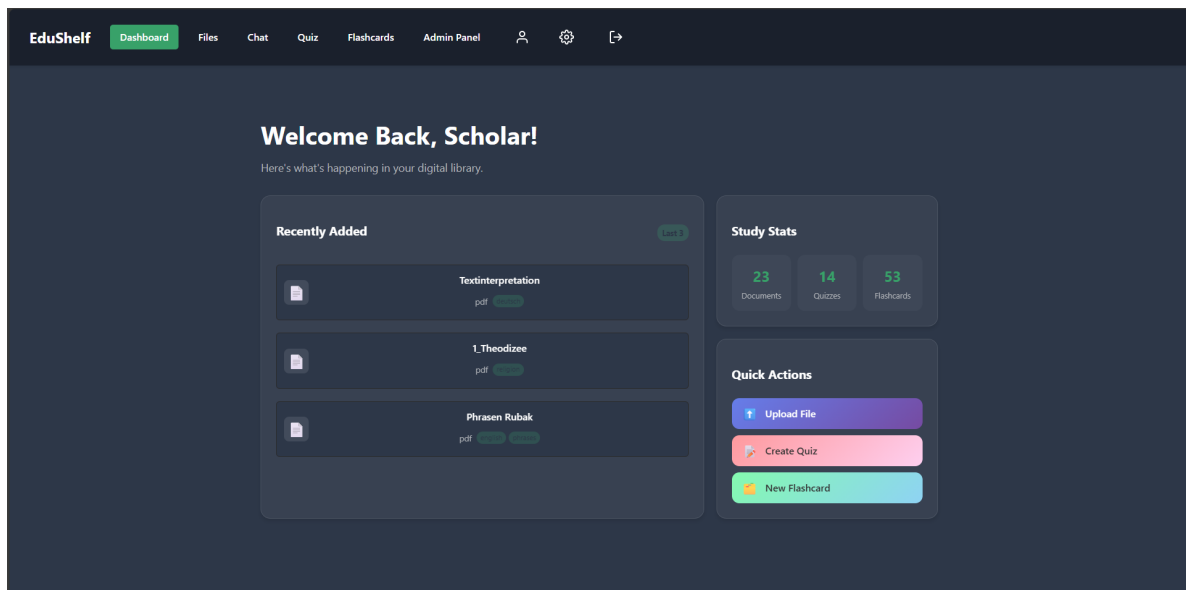


Abbildung 5.1: Hauptdashboard von EduShelf im Dark-Mode

5.2.1 Architektur und Datenladen

Ein zentrales Designprinzip des Dashboards ist das parallele Laden aller benötigten Daten. Anstatt Dokumente, Quiz-Einträge und Lernkarten sequenziell abzurufen – was die Ladezeit mit jeder weiteren Anfrage erhöhen würde – wird `Promise.all()` eingesetzt. Diese Methode startet alle asynchronen Anfragen gleichzeitig und wartet, bis die langsamste von ihnen abgeschlossen ist.

```

1 const fetchData = async () => {
2   try {
3     const [docsData, quizzes, flashcards] = await Promise.all([
4       getDocuments(),
5       getQuizzes(),
6       getFlashcards()
7     ]);
8     // Alle Daten sind gleichzeitig verfuegbar
9     setStats({

```

```

10     documents: docsData.totalCount || docs.length,
11     quizzes:   quizzes.totalCount  || quizzes.length,
12     flashcards: flashcards.totalCount || flashcards.length
13   });
14   } catch (error) {
15     console.error('Error fetching dashboard data:', error);
16   }
17 };

```

Listing 5.1: Paralleles Datenladen mit Promise.all im Dashboard

Das Ergebnis ist eine messbar kürzere Wartezeit im Vergleich zu sequenziellen Aufrufen, da die Gesamtdauer der Nettozeit der langsamsten Einzelanfrage entspricht, nicht der Summe aller Anfragen.

5.2.2 Filterung geteilter Dateien

Das Dashboard implementiert eine clientseitige Filterlogik für geteilte Dateien. Da Dokumente, die von anderen Nutzern geteilt wurden, zunächst im Zustand *ausstehend* verbleiben, werden nur jene Dokumente in der Übersicht angezeigt, die der Nutzer explizit akzeptiert hat. Die Entscheidungen des Nutzers (akzeptiert oder abgelehnt) werden im `localStorage` persistiert und beim Laden ausgewertet:

```

1  const acceptedShares = JSON.parse(
2    localStorage.getItem('edushelf_accepted_shares') || '[]'
3  );
4  const rejectedShares = JSON.parse(
5    localStorage.getItem('edushelf_rejected_shares') || '[]'
6  );
7
8  docs = docs.filter(doc => {
9    if (!doc.isShared) return true;           // eigene Dateien immer anzeigen
10   if (rejectedShares.includes(doc.id)) return false; // abgelehnte
11   //          ausblenden
12   return acceptedShares.includes(doc.id); // nur akzeptierte zeigen
13 });

```

Listing 5.2: Filterung von Shared Files anhand des localStorage-Status

5.2.3 Schnellaktionen und Statistik-Widget

Das Dashboard gliedert sich in drei funktionale Bereiche:

1. **Recently Added:** Zeigt die drei zuletzt hochgeladenen Dokumente mit Dateiname, Typ und zugewiesenen Tags an. Ein Klick öffnet die Inline-Vorschau über `FileViewer`.

2. **Study Stats:** Zeigt aggregierte Zählwerte für Dokumente, Quizzes und Lernkarten. Diese Statistiken werden bei jeder Aktion (Upload, Erstellen, Löschen) durch erneuten Aufruf von `fetchData()` aktualisiert.
3. **Quick Actions:** Drei Schaltflächen ermöglichen das unmittelbare Erstellen eines Uploads, eines Quiz oder einer Lernkarte, ohne dass dafür die entsprechende Unterseite aufgerufen werden muss. Die zugehörigen Modaldialoge werden direkt im Dashboard-Kontext gerendert.

Dieses Konzept minimiert die Anzahl der notwendigen Navigationsschritte für häufig ausgeführte Aktionen und entspricht der Zielsetzung einer effizienten Lernoberfläche.

5.3 Authentifizierungsflow: Registrierung und E-Mail-Bestätigung

Der Authentifizierungsprozess von EduShelf umfasst drei aufeinander aufbauende Schritte: die Registrierung eines neuen Benutzerkontos, die Verifikation der E-Mail-Adresse sowie den eigentlichen Login. Diese Abfolge stellt sicher, dass nur Nutzer mit einer verifizierten Identität Zugang zur Plattform erhalten.

5.3.1 Registrierung (Register.jsx)

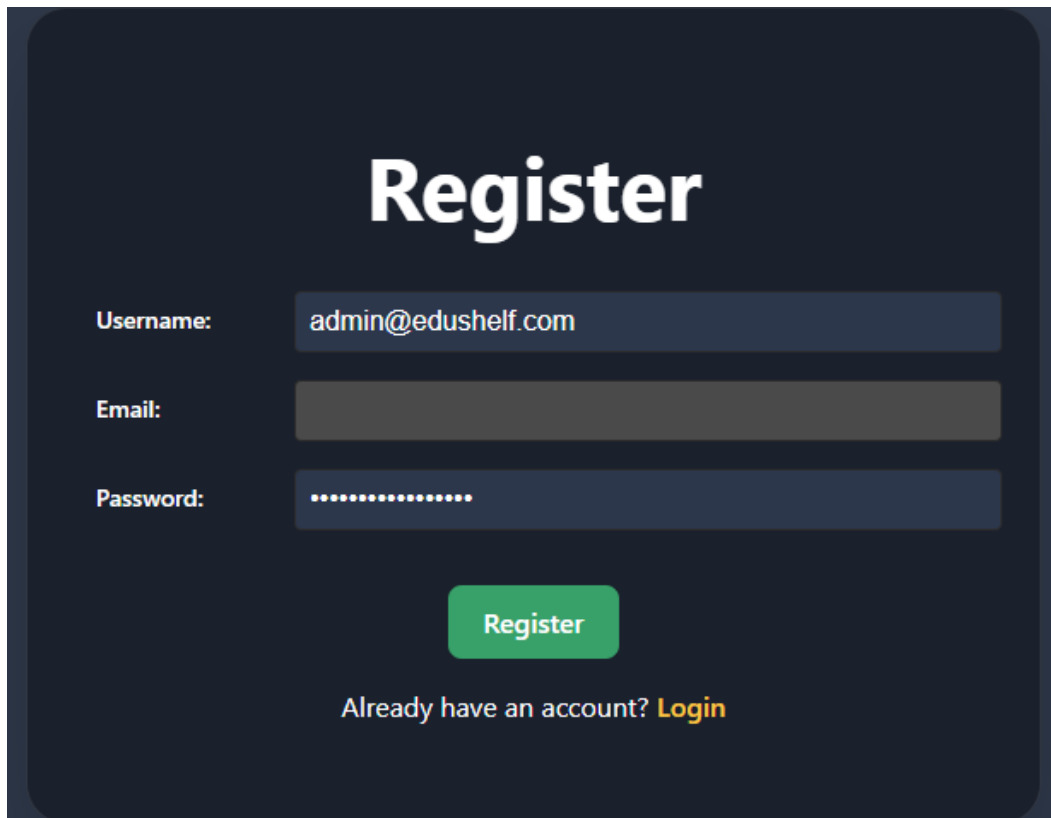


Abbildung 5.2: Registrierungsformular von EduShelf im Dark-Mode

Die Komponente `Register.jsx` realisiert ein zweistufiges Validierungskonzept. Zunächst wird die Benutzereingabe vollständig clientseitig geprüft, bevor eine Anfrage an den Server gesendet wird. Dies reduziert unnötige Netzwerkanfragen und gibt dem Nutzer sofortiges Feedback.

5.3.1.1 Clientseitige Validierung

Die Validierungsfunktion überprüft alle Felder nach klar definierten Regeln:

```
1 const validate = () => {  
2   const newErrors = {};  
3   if (!username || username.length < 3 || username.length > 20) {  
4     newErrors.username = 'Username must be between 3 and 20 characters.';  
5   }  
6   if (!email || !/\S+@\S+\.\S+/.test(email)) {  
7     newErrors.email = 'A valid email address is required.';  
8   }  
9   if (!password || password.length < 6) {  
10    newErrors.password = 'Password must be at least 6 characters long.';  
11  }  
12  setErrors(newErrors);  
}
```

```
13 |   return newErrors;
14 | };
```

Listing 5.3: Clientseitige Eingabevalidierung im Registrierungsformular

Diese Methode setzt auf reguläre Ausdrücke zur E-Mail-Validierung und explizite Längenprüfungen für Benutzername und Passwort. Nur wenn `validate()` ein leeres Fehlerobjekt zurückgibt (d.h. `Object.keys(validationErrors).length === 0`), wird die Backend-Anfrage ausgelöst.

5.3.1.2 Serverseitige Fehlerverarbeitung

Serverseitige Fehler (z. B. ein bereits verwendeter Benutzername) werden strukturiert verarbeitet. Das Backend sendet ein Fehlerobjekt mit feldspezifischen Nachrichten zurück, welches das Frontend auflöst und den entsprechenden Feldern zuordnet:

```
1 if (error.response?.data?.errors) {
2   const backendErrors = {};
3   for (const key in error.response.data.errors) {
4     // Schlüssel mit Kleinbuchstaben normalisieren (z.B. "Username" -> "
       username")
5     const formattedKey = key.charAt(0).toLowerCase() + key.slice(1);
6     backendErrors[formattedKey] = error.response.data.errors[key].join(' ');
7   }
8   setErrors(backendErrors);
9 }
```

Listing 5.4: Verarbeitung von Backend-Validierungsfehlern

Diese Normalisierung des Schlüssels (erster Buchstabe zu Kleinbuchstaben) ist notwendig, da das Backend C#-Konventionen mit großem Anfangsbuchstaben verwendet (`Username`), während React-State-Schlüssel der JavaScript-Konvention (`camelCase`) folgen.

5.3.2 Login-Prozess (Login.jsx)

Der Login-Prozess ist bewusst zweigliedrig gestaltet: Nach dem erfolgreichen POST-Request an `/Users/login` wird unmittelbar ein zweiter Aufruf an `/Users/{id}/roles` durchgeführt. Die Rollenzuweisung ist für die Steuerung der Navigation essenziell – nur Nutzer mit der Rolle *Admin* sehen den Menüpunkt zum Administrationsbereich.

Das zusammengesetzte Benutzerobjekt (Nutzerdaten + Rollen) wird im `localStorage` abgelegt, um den Anmeldestatus über Browser-Freshes hinweg

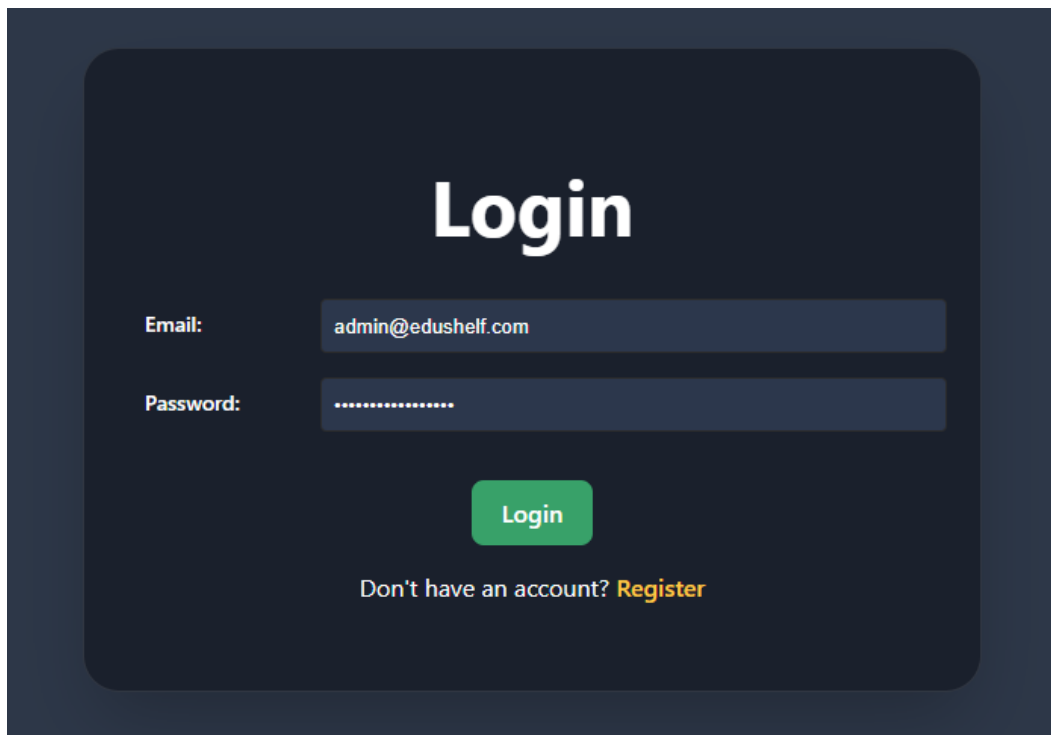


Abbildung 5.3: Login-Ansicht von EduShelf im Dark-Mode

zu erhalten. Diese Persistierung gilt ausschließlich der UI-Zustandsverwaltung; die tatsächliche Sitzungsauthentifizierung erfolgt serverseitig über ein HttpOnly-Cookie.

5.3.3 E-Mail-Bestätigung

Nach der Registrierung erhält der Nutzer eine Bestätigungs-E-Mail. Der enthaltene Link führt zur Komponente `VerifyEmail.jsx`, welche den Token aus der URL extrahiert und an den Backend-Endpunkt zur Verifikation übermittelt. Da das Backend den Link selbst generiert, enthält das Frontend lediglich die Anzeigelogik für die Erfolgs- bzw. Fehlermeldung – eine klare Umsetzung des Single-Responsibility-Prinzips.

5.4 Navigationsstruktur und Layout (MainLayout.jsx)

Das Grundgerüst der Anwendung wird durch die Komponente `MainLayout.jsx` definiert. Sie fungiert als persistente Hülle um alle geschützten Seiten und stellt

sicher, dass die Navigationsleiste stets sichtbar bleibt, während sich der Inhaltsbereich dynamisch verändert.

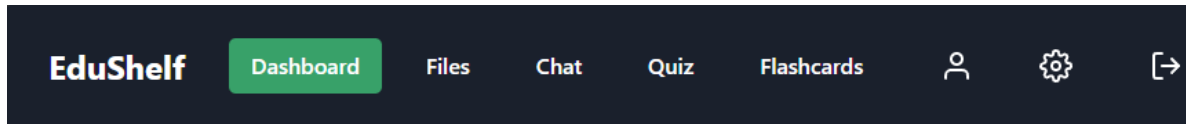


Abbildung 5.4: Navigationsleiste von EduShelf mit rollenbasierter Anzeige (Dark-Mode)

5.4.1 Outlet-Pattern und verschachtelte Routen

Das Layout nutzt das `<Outlet />`-Konzept von React Router. Dieses Muster erlaubt es, die Navigationsleiste und den Inhaltsbereich strikt voneinander zu trennen: Das Layout wird einmalig gerendert und bleibt stabil, während React Router die jeweils aktive Kind-Komponente an der `<Outlet />`-Stelle einfügt. Dadurch werden unnötige Re-Renders der gesamten Navigationsstruktur vermieden.

```

1 const MainLayout = ({ handleLogout, user }) => {
2   return (
3     <div className="app-container">
4       <header className="navbar">
5         { /* Navigationsleiste -- wird nie neu gerendert */ }
6         <nav className="navbar-nav">
7           <NavLink to="/">Dashboard</NavLink>
8           <NavLink to="/files">Files</NavLink>
9           { /* ... */ }
10        </nav>
11      </header>
12      <main className="main-content">
13        <Outlet /> { /* Hier wird die aktive Seite eingefuegt */ }
14      </main>
15    </div>
16  );
17 };

```

Listing 5.5: Outlet-Pattern in MainLayout.jsx

5.4.2 Rollenbasierte Navigation (RBAC)

Die Sichtbarkeit des Menüpunkts *Admin Panel* wird direkt in der Navigationskomponente gesteuert. Ein bedingtes Rendering prüft, ob das übergebene `user`-Objekt die Rolle `'Admin'` im Rollen-Array enthält:

```

1 {user && user.roles && user.roles.includes('Admin')} && (

```

```

2 <NavLink to="/admin">Admin Panel</NavLink>
3 })

```

Listing 5.6: Rollenbasierte Anzeige des Admin-Menüpunkts

Diese Überprüfung findet ausschließlich auf der Client-Seite statt und dient der Benutzerfreundlichkeit. Die eigentliche Zugriffskontrolle auf admin-spezifische API-Endpunkte erfolgt serverseitig, sodass ein manuelles Manipulieren der URL durch den Nutzer keine sicherheitsrelevanten Auswirkungen hat.

5.4.3 Aktiver Zustand mit NavLink

React Router stellt die Komponente `NavLink` bereit, welche gegenüber dem einfachen `Link` den Vorteil bietet, den aktuell aktiven Zustand zu kennen. Über eine Callback-Funktion kann die CSS-Klasse des Elements dynamisch gesetzt werden:

```

1 <NavLink
2   to="/files"
3   className={({ isActive }) => (isActive ? 'active' : '')}
4 >
5   Files
6 </NavLink>

```

Listing 5.7: Dynamische CSS-Klasse bei aktivem Navigationselement

Der Parameter `isActive` wird von React Router automatisch auf `true` gesetzt, wenn die aktuelle URL mit dem `to`-Attribut des Links übereinstimmt. Die CSS-Klasse `active` hebt den entsprechenden Menüpunkt visuell hervor und verbessert die räumliche Orientierung des Nutzers innerhalb der Anwendung.

5.5 Datei-Sharing zwischen Nutzern

Eine wesentliche kollaborative Funktion von EduShelf ist die Möglichkeit, Dokumente mit anderen Nutzern zu teilen. Der gesamte Sharing-Workflow wird durch die Komponente `ShareFileModal.jsx` in Verbindung mit der Dateiansicht `Files.jsx` realisiert.

5.5.1 Share-Dialog (`ShareFileModal.jsx`)

Der Share-Dialog ist als kontrolliertes Formular implementiert. Der Nutzer gibt die E-Mail-Adresse oder den Benutzernamen der Zielperson ein. Da beide Ein-

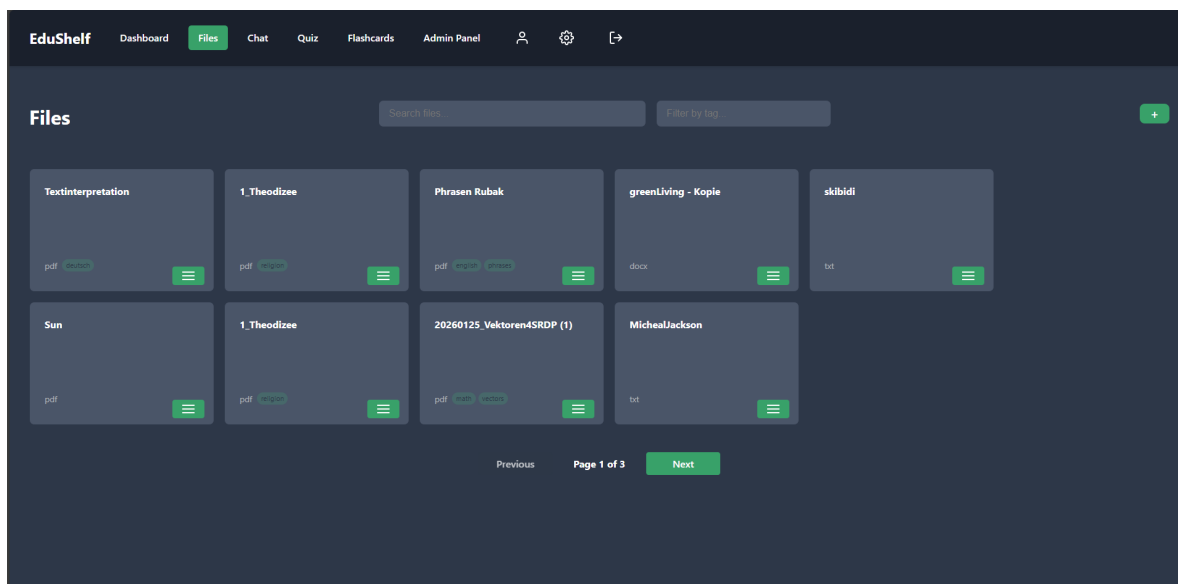


Abbildung 5.5: Dateiansicht von EduShelf (Dark-Mode)

gabeformate akzeptiert werden, entfällt die Notwendigkeit, die genaue Identifikation des Empfängers zu kennen – das Backend löst die Zuordnung auf.

```

1  const ShareFileModal = ({ isOpen, onClose, onShare, fileName }) => {
2    const [emailOrUsername, setEmailOrUsername] = useState('');
3    const [isLoading, setIsLoading] = useState(false);
4
5    const handleSubmit = async (e) => {
6      e.preventDefault();
7      if (!emailOrUsername.trim()) return;
8
9      setIsLoading(true);
10     try {
11       await onShare(emailOrUsername); // Callback an Files.jsx
12       setEmailOrUsername('');         // Eingabe nach Erfolg
13       // zuruecksetzen
14     } finally {
15       setIsLoading(false);
16     }
17   };
18 };

```

Listing 5.8: Formularlogik im ShareFileModal

Während des Sendevorgangs wird der Submit-Button deaktiviert (`disabled=isLoading`) und der Beschriftungstext wechselt von *Share File* zu *Sharing...*. Dieses Muster verhindert Mehrfach-Submissions und gibt dem Nutzer klares visuelles Feedback über den laufenden Prozess.

5.5.2 Zustandsverwaltung für Shared Files

Empfangene Freigaben erscheinen im Dateibereich des Empfängers mit einem visuellen Kennzeichen (`isShared: true`). Der Nutzer hat die Möglichkeit, eine Freigabe zu akzeptieren oder abzulehnen. Diese Entscheidung wird clientseitig im `localStorage` persistiert, da sie ausschließlich die Sichtbarkeit in der UI steuert:

```

1 // Akzeptieren einer Freigabe
2 const handleAcceptShare = (docId) => {
3   const accepted = JSON.parse(
4     localStorage.getItem('edushelf_accepted_shares') || '[]'
5   );
6   localStorage.setItem(
7     'edushelf_accepted_shares',
8     JSON.stringify([...accepted, docId])
9   );
10  fetchFiles(); // Listenaktualisierung auslösen
11 };
12
13 // Ablehnen einer Freigabe
14 const handleRejectShare = (docId) => {
15   const rejected = JSON.parse(
16     localStorage.getItem('edushelf_rejected_shares') || '[]'
17   );
18   localStorage.setItem(
19     'edushelf_rejected_shares',
20     JSON.stringify([...rejected, docId])
21   );
22   fetchFiles();
23 };

```

Listing 5.9: Persistierung der Share-Entscheidungen im `localStorage`

5.5.3 Bewertung des Ansatzes

Die Entscheidung, den Accept/Reject-Status clientseitig zu speichern, hat sowohl Vor- als auch Nachteile. Der wesentliche Vorteil ist die sofortige Reaktivität ohne serverseitigen Roundtrip. Der Nachteil besteht in der fehlenden Geräteübergreifenden Synchronisation: Akzeptiert ein Nutzer eine Datei auf einem Gerät, ist sie auf einem anderen Gerät weiterhin als ausstehend markiert. Für eine zukünftige Version wäre eine serverseitige Persistierung des Share-Status daher erstrebenswert.

5.6 Technologie-Stack

Die Auswahl der Technologien erfolgte unter den Gesichtspunkten der Modernität, Verbreitung (Community-Support) und Wartbarkeit.

5.6.1 React (v19)

React ist eine von Facebook (Meta) entwickelte JavaScript-Bibliothek zur Erstellung von Benutzeroberflächen. Im Kern basiert React auf Komponenten, die wiederverwendbare Bausteine der UI darstellen. Ein Schlüsselement von React ist das „Virtual DOM“. Anstatt bei jeder Änderung direkt das langsame, reale DOM des Browsers zu manipulieren, erstellt React eine virtuelle Kopie im Speicher. Bei Zustandsänderungen vergleicht React den neuen virtuellen Baum mit dem alten (Diffing-Algorithmus) und berechnet die effizienteste Methode, um das reale DOM zu aktualisieren.

In EduShelf wird ausschließlich die „Functional Component“ Syntax in Verbindung mit „Hooks“ verwendet. Hooks wie `useState` oder `useEffect` ermöglichen es, Zustand und Seiteneffekte in funktionalen Komponenten zu nutzen, ohne Klassen schreiben zu müssen. Dies führt zu kompakterem und verständlicherem Code.

5.6.2 Vite

Vite (französisch für „schnell“) ist das Build-Tool der Wahl für „EduShelf“. Es löst ältere Tools wie Webpack ab, indem es während der Entwicklung native ES-Module im Browser nutzt. Das bedeutet, dass der Browser die Importe selbst auflöst, anstatt dass der gesamte Code vor jedem Start gebündelt werden muss. Dies führt zu extrem schnellen Server-Startzeiten, selbst bei wachsender Projektgröße. Für die Produktion nutzt Vite im Hintergrund Rollup, um hochoptimierte, minifizierte Bundles zu erstellen.

5.6.3 React Router (v7)

Da eine SPA (= Single Page Application) physisch nur aus einer Seite besteht, muss die Navigation simuliert werden. **React Router** überwacht die URL im Browser und rendert basierend auf definierten Regeln die entsprechenden Komponenten. Dies ermöglicht Deep-Linking (Lesezeichen auf Unterseiten) und die

Verwendung der Browser-Historie (Zurück-Button), obwohl die Seite nie neu geladen wird.

5.7 Projektsetup und Konfiguration

Das Projekt wurde mit dem Befehl `npm create vite@latest` initialisiert. Die Konfiguration befindet sich in der Datei `vite.config.js`.

```
1 import { defineConfig } from 'vite'
2 import react from '@vitejs/plugin-react'
3
4 // https://vite.dev/config/
5 export default defineConfig({
6   plugins: [react()],
7   server: {
8     host: true,
9     port: 5173,
10    proxy: {
11      '/api': {
12        target: process.env.VITE_API_TARGET || 'http://localhost:5015',
13        changeOrigin: true
14      }
15    }
16  }
17 })
```

Listing 5.10: Auszug aus der Vite-Konfiguration

Ein kritischer Aspekt dieser Konfiguration ist der **Proxy**: Um während der Entwicklung Cross-Origin Resource Sharing (CORS) Probleme zu vermeiden, leitet der Vite-Entwicklungsserver alle Anfragen, die mit `/api` beginnen, an das Backend (Port 5015) weiter. Die Option `changeOrigin: true` manipuliert dabei den Host-Header der Anfrage so, als käme sie direkt vom Backend-Server selbst.

Die Projektstruktur im Order `src` folgt einer funktionalen Aufteilung:

- `components/`: Beinhaltet UI-Komponenten (z.B. `Chat.jsx`, `Quiz.jsx`). Jede Komponente hat idealerweise ihre eigene CSS-Datei (z.B. `Chat.css`) für isoliertes Styling.
- `services/`: Kapselt die gesamte API-Kommunikation.
- `assets/`: Für statische Dateien.

5.8 Softwarearchitektur und Datenfluss

5.8.1 Unidirektionaler Datenfluss

React erzwingt einen unidirektionalen Datenfluss („One-Way Binding“). Daten fließen immer von Eltern- zu Kind-Komponenten mittels sogenannter „Props“. Um Daten von einem Kind zurück zum Elternteil zu senden (z.B. ein Formular-Submit im `QuizModal`, der die Liste im `Quiz` aktualisiert), übergibt die Elternkomponente eine Callback-Funktion an das Kind.

5.8.2 Routing-Strategie

Das Routing in `App.jsx` definiert die Skelettstruktur der Anwendung. Es wird zwischen öffentlichen Routen (Login, Register) und privaten, geschützten Routen unterschieden.

```

1 <Routes>
2   { /* Öffentliche Routen */ }
3   <Route path="/login" element={<Login setLoggedInUser={setLoggedInUser} />}
4     />
5   { /* Geschuetzte Routen - erfordern Login */ }
6   <Route path="/*" element={
7     loggedInUser ? (
8       <Routes>
9         <Route element={<MainLayout handleLogout={handleLogout} user={
10           loggedInUser} />} />
11         <Route index element={<MainDashboard />} />
12         <Route path="files" element={<Files />} />
13         <Route path="chat" element={<Chat />} />
14         { /* ... weitere Routen ... */ }
15       </Routes>
16     ) : (
17       <Navigate to="/login" />
18     )
19   } />
20 </Routes>

```

Listing 5.11: Routing-Struktur in `App.jsx`

Die Verwendung von verschachtelten Routen (`<Route element=<MainLayout ...>>`) erlaubt es, das Grundgerüst (Header, Navigation) einmalig zu definieren. Die Kind-Komponenten werden dann an der Stelle des `<Outlet />` Platzhalters in `MainLayout.jsx` gerendert.

5.9 Datenschicht und API-Integration

Die gesamte Kommunikation mit dem Backend wird über das Modul `src/services/api.js` abgewickelt. Dieses Muster, oft als "SService Layer Pattern" bezeichnet, hat den Vorteil, dass Komponenten keine Details über URLs oder HTTP-Header kennen müssen.

5.9.1 API-Wrapper

Das `api`-Objekt fungiert als Wrapper um die native `fetch`-API. Es implementiert globale Verhaltensweisen:

1. **Basis-URL:** Alle Anfragen sind relativ zu einer zentral konfigurierten Basis-URL.
2. **Credentials:** Durch `credentials: 'include'` sendet der Browser automatisch Cookies (für die `HttpOnly` Session-Cookies) mit jeder Anfrage mit.
3. **Content-Type Handling:** JSON-Payloads werden automatisch serialisiert und der korrekte Header gesetzt.
4. **Zentrale Fehlerbehandlung:** Antworten mit einem HTTP-Statuscode außerhalb des 200er-Bereichs lösen automatisch eine Exception aus. Dies vereinfacht die Fehlerbehandlung in den Komponenten, da `try/catch`-Blöcke verwendet werden können.

```

1 get: async (endpoint, options = {}) => {
2   const response = await fetch(`${API_BASE_URL}${endpoint}`, {
3     credentials: 'include',
4     ...options,
5   });
6   if (!response.ok) {
7     // Fehlerbehandlung: Logging und Werfen eines Errors
8     throw new Error('API Error: ${response.status}');
9   }
10  return await response.json();
11 },

```

Listing 5.12: Generische GET-Methode im API-Service

Zusätzlich stellt der Service spezifische Methoden bereit, die die Geschäftslogik abbilden, beispielsweise `createChatSession`, `updateQuiz` oder `uploadDocument`. Letzteres nutzt `FormData`, um Datei-Uploads (multipart/form-data) zu ermöglichen.

5.10 Implementierung der Kernkomponenten

5.10.1 Authentifizierungssystem

Das Login-System (`Login.jsx`) ist der erste Berührungspunkt für den Nutzer. Das Formular sendet E-Mail und Passwort an den Endpunkt `/Users/login`. Bei Erfolg antwortet das Server-Backend mit einem Session-Cookie (`HttpOnly`) und den Benutzerdaten.

Das Besondere an der Implementierung ist das sofortige Abrufen der Benutzerrollen nach dem Login. Da die Rollen („Admin“, „User“) bestimmen, welche Menüpunkte sichtbar sind, führt das Frontend einen sekundären Aufruf an `/Users/{id}/roles` durch. Das kombinierte Benutzerobjekt wird anschließend im `localStorage` gespeichert. Dies dient nicht der Sicherheit (da der `localStorage` manipuliert werden kann), sondern lediglich der Persistenz des UI-Zustands (z. B. „Ist der Nutzer eingeloggt?“), bis die echte Session beim Neuladen durch `/Users/me` validiert wird.

5.10.2 Dateimanagement (`Files.jsx`)

Die Komponente `Files.jsx` realisiert ein Dokumentenmanagementsystem.
Funktionsweise:

- Beim Laden der Komponente ruft `useEffect` die Liste aller Dokumente ab.
- Ein Suchfeld filtert die lokale Liste in Echtzeit (`filteredFiles.filter(...)`).
- Das Kontextmenü (Drei-Punkte-Menü) wird über den Zustand `activeMenuId` gesteuert, sodass immer nur ein Menü gleichzeitig offen sein kann.

Der Upload-Prozess wird über ein modales Fenster (`UploadDialog.jsx`) realisiert. Hier wählt der Nutzer eine Datei aus, welche zusammen mit der User-ID an den Server gesendet wird.

5.10.3 Interaktives Lernen: Das Quiz-Modul

Das Quiz-System ist eine der komplexesten Komponenten des Frontends. Es besteht aus der Übersichtsliste (`Quiz.jsx`) und dem Editor/Player (`QuizModal.jsx`, `Quiz.jsx` Logik).

5.10.3.1 Datenstruktur

Ein Quiz besteht aus einem Titel und einer Liste von Fragen. Jede Frage enthält wiederum eine Liste von Antworten, wobei eine oder mehrere als korrekt markiert sind. Diese hierarchische Struktur wird im State als verschachteltes Array objekt abgebildet.

5.10.3.2 Ablauf eines Quiz

Der Zustand während eines laufenden Quiz wird durch mehrere State-Variablen gesteuert:

- `currentQuestionIndex`: Index der aktuell angezeigten Frage.
- `score`: Anzahl der bisher korrekt beantworteten Fragen.
- `selectedAnswerId`: Um eine Mehrfachauswahl zu verhindern und visuelles Feedback zu geben.
- `showScore`: Ein Boolean-Flag, das nach der letzten Frage auf `true` gesetzt wird, um die Ergebnisanzeige zu rendern.

Das visuelle Feedback (Grün für richtig, Rot für falsch) wird durch CSS-Klassen gesteuert, die dynamisch basierend auf der Korrektheit der Antwort zugewiesen werden.

5.10.4 Flashcards (Lernkarten)

Die `Flashcards.jsx` Komponente nutzt CSS3 3D-Transformationen, um einen realistischen Umdreh-Effekt zu erzielen.

```

1 .flashcard-scene {
2   perspective: 1000px; /* Gibt dem 3D-Raum Tiefe */
3 }
4 .flashcard-container {
5   transition: transform 0.6s;
6   transform-style: preserve-3d;

```

```
7 | }  
8 | .flashcard-scene.flipped .flashcard-container {  
9 |   transform: rotateY(180deg);  
10| }
```

Listing 5.13: CSS für den Flip-Effekt

Der Status, welche Karte gerade umgedreht ist, wird in einem `Set` (`flippedCards`) gespeichert. Dies ermöglicht es, mehrere Karten gleichzeitig umzudrehen, im Gegensatz zu einer simplen ID-Variable.

5.10.5 Chat und KI-Integration

Der Chat (`Chat.jsx`) bietet eine Schnittstelle zu einem LLM (Large Language Model). Wichtige Implementierungsdetails:

- **Streaming vs. Polling:** Die aktuelle Implementierung wartet auf die vollständige Antwort des Servers (Request/Response Pattern). Während des Wartens wird ein "Thinking Indicator" (animierte Punkte) eingeblendet (`isLoading` State).
- **Markdown:** Da KI-Modelle oft strukturierten Text (Codeblöcke, Listen) ausgeben, wird `react-markdown` in Kombination mit `rehype-highlight` verwendet. Dies konvertiert die Markdown-Antwort sicher in HTML und bietet Syntax-Highlighting für Code.
- **Multimodaler Input:** Benutzer können Bilder hochladen. Diese werden lokal mittels `URL.createObjectURL()` sofort in der Vorschau angezeigt, bevor sie als Blob an den Server gesendet werden.

5.10.6 Admin Panel und RBAC

Das `AdminPanel.jsx` ist das Kontrollzentrum für Systemadministratoren. Der Zugriff auf diese Route wird auf zwei Ebenen geschützt: 1. **Client-Side Routing:** In `App.jsx` wird geprüft, ob das Benutzerobjekt die Rolle `Admin` enthält. Falls nicht, wird der Nutzer sofort umgeleitet. 2. **Server-Side API:** Selbst wenn ein Nutzer die UI umgehen würde, lehnt das Backend Anfragen an admin-spezifische Endpunkte ab.

Das Panel ermöglicht CRUD-Operationen (Create, Read, Update, Delete) für Benutzer. Besonders hervorzuheben ist die verschachtelte Datenansicht: Klickt ein Admin auf einen Benutzer, werden dessen Dateien asynchron nachgeladen

(`fetchUserFiles`) und in einer Detailansicht dargestellt. Dies vermeidet das Laden unnötiger Daten für die Hauptübersichtstabelle.

5.11 Benutzeroberfläche und Design (UI/UX)

5.11.1 Styling-Strategie

Das Projekt verwendet natives CSS mit CSS-Variablen (Custom Properties) für das Theming. Dies bietet maximale Flexibilität ohne den Overhead großer Frameworks. Globale Styles sind in `index.css` definiert, während komponentenspezifische Styles (z. B. `Chat.css`) lokal importiert werden.

5.11.2 Dark Mode Implementierung

Der Dunkelmodus ist nicht nur eine kosmetische Spielerei, sondern eine zentrale Anforderung. Er wurde über CSS-Variablen realisiert, die je nach Attribut am HTML-Tag ihre Werte ändern.

```

1 :root {
2   /* Standard (Dark Mode) */
3   --primary-color: #222831;
4   --text-color: #FFFFFF;
5 }
6
7 [data-theme='light'] {
8   /* Light Mode Overrides */
9   --primary-color: #FFFFFF;
10  --text-color: #222831;
11 }
12
13 body {
14   background-color: var(--primary-color);
15   color: var(--text-color);
16 }

```

Listing 5.14: Theme-Switching mittels CSS-Variablen

Ein `useEffect`-Hook in der `App.jsx` prüft beim Start den `localStorage`, ob der Nutzer eine Präferenz gespeichert hat, und setzt das Attribut entsprechend. Dies verhindert den Flash of Incorrect Theme"(FOUC).

5.12 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe der Implementierung zu verdeutlichen, werden im Folgenden drei Kernmodule detailliert anhand ihres Quellcodes analysiert.

5.12.1 Quiz-Editor Logik (QuizModal.jsx)

Der Quiz-Editor ermöglicht das Erstellen und Bearbeiten von verschachtelten Datenstrukturen (Quiz -> Fragen -> Antworten). Die Herausforderung hierbei ist das State-Management einer mehrdimensionalen Struktur.

5.12.1.1 Zustandsverwaltung

Der State `questions` ist ein Array von Objekten. Jedes Objekt repräsentiert eine Frage und enthält wiederum ein Array von `answers`.

```
1 const [questions, setQuestions] = useState([  
2   {  
3     text: '',  
4     answers: [{ text: '', isCorrect: false }]  
5   }  
6 ]);
```

Listing 5.15: Initialer State im QuizModal

5.12.1.2 Dynamische Formular-Manipulation

Beim Ändern eines Antworttextes darf der State nicht direkt mutiert werden. Stattdessen muss eine tiefe Kopie (Deep Copy) oder zumindest eine flache Kopie der betroffenen Ebenen erstellt werden. Die Funktion `handleAnswerChange` demonstriert das Prinzip der Immutability in React:

```
1 const handleAnswerChange = (questionIndex, answerIndex, event) => {  
2   // 1. Kopie des Fragen-Arrays  
3   const newQuestions = [...questions];  
4   // 2. Direkter Zugriff auf die Antwort (Referenz bleibt erhalten, aber  
5     // Inhalt wird ueberschrieben)  
6   // Hinweis: Fuer saubere Immutability muesste auch answers kopiert  
7     // werden.  
8   // In React funktioniert dies jedoch oft, solange das Top-Level-Array  
9     // neu ist.  
10  newQuestions[questionIndex].answers[answerIndex].text = event.target.  
11    value;
```



```

8 // 3. Setzen des neuen States löst Re-Render aus
9 setQuestions(newQuestions);
10 };

```

Listing 5.16: Update-Logik für verschachtelte Antworten

5.12.1.3 Validierung und Speichern

Beim Speichern (`handleSubmit`) werden die Daten an die API gesendet. Hierbei wird unterschieden, ob ein neues Quiz erstellt (POST) oder ein bestehendes bearbeitet (PUT) wird.

5.12.2 Flashcards: 3D-Interaktion und State

Die Komponente `Flashcards.jsx` verwendet eine interessante Datenstruktur für den Zustand der umgedrehten Karten. Anstatt für jede Karte einen Boolean im Objekt zu speichern (was eine Änderung des Datenmodells erfordern würde), wird ein `Set` verwendet, das die IDs der umgedrehten Karten speichert.

```

1 const [flippedCards, setFlippedCards] = useState(new Set());
2
3 const flipCard = (id) => {
4   setFlippedCards(prevFlippedCards => {
5     const newFlippedCards = new Set(prevFlippedCards);
6     if (newFlippedCards.has(id)) {
7       newFlippedCards.delete(id); // Karte umdrehen (zurueck)
8     } else {
9       newFlippedCards.add(id); // Karte aufdecken
10    }
11    return newFlippedCards;
12  });
13 };

```

Listing 5.17: Nutzung eines Sets fuer UI-State

Diese Methode ist performant und trennt den UI-Zustand (ist die Karte gedreht?) sauber von den eigentlichen Geschäftsdaten (Frage/Antwort). Beim Rendern wird `flippedCards.has(card.id)` geprüft, um die CSS-Klasse `.flipped` bedingt hinzuzufügen.

5.12.3 Admin Panel: Kaskadierende Datenabfragen

Im `AdminPanel.jsx` muss beim Klick auf einen Benutzer dessen Dateiliste geladen werden. Dies ist ein klassisches Master-DetailSzenario. Um unnötigen

Netzwerktraffic zu vermeiden, werden die Dateien nicht initial für alle Benutzer geladen (Lazy Loading").

```
1 const handleClick = (user) => {
2   setSelectedUser(user);
3   setError('');
4   setEditMode(false);
5   // Erst beim Klick werden die Dateien nachgeladen
6   fetchUserFiles(user.userId);
7 };
8
9 const fetchUserFiles = async (userId) => {
10   try {
11     setLoading(true); // Ladeindikator an
12     const files = await api.get(`/Users/${userId}/documents`);
13     setUserFiles(files);
14   } catch (err) {
15     setError('Could not load user files.');
```

Listing 5.18: Asynchrones Laden von Detaildaten

Diese Implementierung sorgt für eine schnelle initiale Ladezeit der Benutzertabelle, da die (potenziell große) Menge an Dateimetadaten erst bei Bedarf (Ön-Demand") abgerufen wird.

5.13 Deployment und Betrieb

Für den produktiven Einsatz wird die Frontend-Anwendung containerisiert. Dies gewährleistet eine konsistente Laufzeitumgebung unabhängig vom Host-System.

5.13.1 Docker-Containerisierung

Es wird ein „Multi-Stage Build“ Prozess verwendet, um das finale Image so klein wie möglich zu halten.

1. **Build-Stage:** Ein Node.js Image wird verwendet, um die Abhängigkeiten zu installieren und den Build-Prozess (`npm run build`) auszuführen. Das Ergebnis ist ein Ordner `dist`, der nur die statischen Dateien (HTML, CSS, JS) enthält.

2. **Production-Stage:** Ein leichtgewichtiger Nginx-Webserver wird als Basis genommen. Die Dateien aus der Build-Stage werden in das Verzeichnis `/usr/share/nginx/html` kopiert.

```
1 # Stage 1: Build
2 FROM node:20-alpine as build
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 # Stage 2: Serve
10 FROM nginx:alpine
11 COPY --from=build /app/dist /usr/share/nginx/html
12 COPY nginx.conf /etc/nginx/conf.d/default.conf
13 EXPOSE 80
14 CMD ["nginx", "-g", "daemon off;"]
```

Listing 5.19: Dockerfile für das Frontend

5.13.2 Nginx Konfiguration

Da es sich um eine SPA handelt, muss der Webserver so konfiguriert werden, dass er für alle Routen (z.B. `/files`, `/chat`) immer die `index.html` ausliefert (Fallback"). Andernfalls würde ein direkter Aufruf einer Unterseite zu einem 404-Fehler führen, da diese Dateien physisch nicht existieren.

5.14 Herausforderungen und Lösungen

Während der Entwicklung traten verschiedene technische Herausforderungen auf, die spezifische Lösungen erforderten.

5.14.1 Synchronisation des UI-Status

Problem: Wenn ein Nutzer ein Quiz bearbeitet, muss die Liste der Quizzes im Hintergrund aktuell bleiben. **Lösung:** Callback-Funktionen (z.B. `onQuizSaved`) werden an die Modal-Komponenten übergeben. Nach erfolgreicher API-Antwort ruft das Modal diese Funktion auf, und die Elternkomponente aktualisiert ihren lokalen State, ohne einen erneuten Fetch auszuführen. Optimistic UI Updates wurden in Betracht gezogen, aber zugunsten der Datenkonsistenz verworfen.

5.14.2 Performance bei großen Dateilisten

Problem: Das Rendern von hunderten Dateien führte zu Verzögerungen. **Lösung:** Durch die Pagination lässt sich dies optimieren.

5.15 Ausblick und Erweiterungsmöglichkeiten

Das aktuelle Frontend bietet eine solide Basis, kann jedoch in Zukunft erweitert werden:

- **Automatisierte Tests:** Integration von `Vitest` für Unit-Tests der Logik und `Cypress` für End-to-End Tests der Benutzeroberfläche.
- **PWA (Progressive Web App):** Durch Hinzufügen eines Service Workers und eines Web App Manifests könnte die Anwendung offline-fähig gemacht und auf Mobilgeräten installiert werden.
- **Internationalisierung (i18n):** Nutzung von Bibliotheken wie `react-i18next`, um die Oberfläche in mehreren Sprachen anzubieten.

5.16 Fazit

Die Frontend-Implementierung von EduShelf demonstriert die Leistungsfähigkeit moderner Webtechnologien. Durch den Einsatz von React und einer komponentenbasierte Architektur entstand eine wartbare, skalierbare und benutzerfreundliche Anwendung. Die Integration komplexer Features wie Real-Time-Editoren, KI-Chat und interaktive Lernmodule in einer Single Page Application zeigt, dass Webanwendungen nativen Programmen in nichts nachstehen müssen.

Zeiterfassung



Abbildung 5.6: AI und iOS Zeitaufwand

Gesamt: 288,67h

Freizeit: 211,38h

6 Backend Implementierung

Die technische Umsetzung des EduShelf-Backends stellt das fundamentale Gerüst der Anwendung dar. Es wurde als monolithische ASP.NET Core Web API konzipiert, die als zentrale Schnittstelle zwischen dem Frontend, der Datenbank, dem KI-Modell und dem Dateispeicherdienst dient. Diese Entscheidung für einen monolithischen Ansatz wurde bewusst getroffen, um die Entwicklungskomplexität initial gering zu halten (*Keep It Simple, Stupid - KISS*), während durch den Einsatz moderner .NET 8 Technologien gleichzeitig eine hohe Performance, Skalierbarkeit und Wartbarkeit gewährleistet wird.

In den folgenden Abschnitten wird die Architektur, der Technologie-Stack, das Datenmodell sowie die spezifischen Implementierungsdetails der Kernkomponenten detailliert erläutert.

6.1 Systemarchitektur

Die Architektur des Systems folgt dem bewährten Muster einer *Layered Architecture* (Schichtenarchitektur), um eine saubere Trennung der Verantwortlichkeiten (*Separation of Concerns*) zu erzielen. Dies erleichtert nicht nur die Testbarkeit mittels Unit- und Integrationstests, sondern ermöglicht auch zukünftige Erweiterungen ohne das Gesamtsystem neu schreiben zu müssen.

Die Anwendung gliedert sich in vier Hauptschichten:

6.1.1 Presentation Layer (Controllers)

Diese oberste Schicht nimmt HTTP-Anfragen (Requests) entgegen und gibt HTTP-Antworten (Responses) zurück. Sie dient als Tor zur Außenwelt.

- **Verantwortlichkeit:** Routing, Deserialisierung von JSON-Body-Daten, Validierung von Eingaben und Formatierung der Ausgaben.

- **Validierung:** Mithilfe der Bibliothek *FluentValidation* werden eingehende Datenmodelle (DTOs) noch vor der Verarbeitung auf Korrektheit geprüft (z.B. `UserRegisterValidator`, der sicherstellt, dass E-Mail-Adressen gültig sind und Passwörter Mindestanforderungen erfüllen).
- **Controller:** Die Controller sind schlank gehalten (*Thin Controllers*) und delegieren die eigentliche Geschäftslogik sofort an die Service-Schicht. Beispiel: Der `ChatController` leitet die Benutzeranfrage direkt an den `ChatService` weiter.

6.1.2 Business Logic Layer (Services)

Hier residiert das "Gehirn" der Anwendung. Alle fachlichen Regeln und Prozesse sind in dieser Schicht implementiert.

- **Service-Klassen:** Jede Domäne hat ihren eigenen Service (z.B. `AuthService`, `IndexingService`, `RAGService`).
- **Orchestrierung:** Services koordinieren den Datenfluss zwischen der Datenbank, externen Diensten (wie MinIO oder Ollama) und der Präsentationsschicht.
- **Dependency Injection:** Alle Services werden über das in .NET integrierte Dependency Injection (DI) Container registriert, was die Austauschbarkeit von Implementierungen zur Laufzeit ermöglicht.
- **Hintergrundverarbeitung:** Für langlaufende Aufgaben wie das Indizieren von Dokumenten wird eine `BackgroundJobQueue` eingesetzt. Dies entkoppelt rechenintensive Prozesse vom HTTP-Request-Zyklus und verhindert Timeouts im Frontend. Der `DocumentService` reiht Aufgaben ein, die dann von einem `BackgroundService` asynchron abgearbeitet werden.

6.1.3 Data Access Layer (Data)

Diese Schicht abstrahiert den Zugriff auf die persistente Datenspeicherung.

- **Entity Framework Core (EF Core):** Als Object-Relational Mapper (ORM) ermöglicht EF Core die Definition des Datenmodells in C#-Klassen. SQL-Queries werden zur Laufzeit generiert (LINQ to SQL), was die Entwicklung beschleunigt und vor SQL-Injection schützt.

- **Repositories:** In diesem Projekt wird das `DbContext`-Pattern direkt verwendet, was bei EF Core oft äquivalent zum Repository-Pattern bzw. Unit-of-Work-Pattern ist.

6.1.4 Infrastructure Layer

Diese Schicht beinhaltet die technische Anbindung an externe Systeme und Infrastrukturkomponenten.

- **MinIO:** Für die Speicherung unstrukturierter Daten (Blobs) wie PDF-Dokumente oder Bilder.
- **Ollama:** Schnittstelle zum lokalen KI-Modell.
- **E-Mail-Dienst:** Implementierung des `SmtpEmailService` zum Versenden von Bestätigungs-E-Mails.

6.2 Technologie-Stack

Das Backend setzt auf einen modernen, vollständig Open-Source-basierten Stack, der speziell für performante, cloud-native und KI-gestützte Anwendungen optimiert wurde.

- **ASP.NET Core 8:** Das zugrundeliegende Framework bietet native Unterstützung für asynchrone Programmierung (`async/await`), was für I/O-lastige Operationen (Datenbank, Netzwerk) essenziell ist. Es ist plattformunabhängig und läuft in Linux-Containern.
- **PostgreSQL + pgvector:** PostgreSQL wurde aufgrund seiner Robustheit und Erweiterbarkeit gewählt. Die entscheidende Komponente für die RAG-Funktionalität ist die Erweiterung `pgvector`. Sie ermöglicht das Speichern von hochdimensionalen Vektoren (Embeddings) direkt in der Datenbank und führt Ähnlichkeitssuchen (Nearest Neighbor Search) mittels Kosinus-Distanz oder Euklidischer Distanz performant durch.
- **Ollama:** Ein lokaler Server zur Ausführung von Large Language Models (LLMs) wie Llama 3 oder Phi-3. Durch die lokale Ausführung bleiben sensible Dokumentendaten auf dem Server und werden nicht an Drittanbieter-APIs (wie OpenAI) gesendet, was den Datenschutz massiv verbessert.

- **MinIO:** Ein S3-kompatibler Object Storage. Hochgeladene Dateien werden nicht als BLOB in der relationalen Datenbank gespeichert (was diese aufblähen würde), sondern effizient im Dateisystem über MinIO verwaltet.
- **Microsoft Semantic Kernel:** Ein SDK, das als Abstraktionsschicht für KI-Dienste dient. Es vereinfacht die Integration verschiedener KI-Modelle und bietet Mechanismen für Prompt-Templating, Memory-Management (Vektorspeicher-Abstraktion) und Function Calling.
- **PdfPig & Docnet.Core:** Spezialisierte Bibliotheken zur Extraktion von Text aus PDF-Dateien. Docnet.Core ist ein Wrapper um die native 'pdfium'-Bibliothek und bietet extrem hohe Performance beim Rendern, während PdfPig reines C# ist und gut für Text-Extraktion geeignet ist.

6.3 Projekt-Setup und Konfiguration

Analog zum Frontend erfolgt der Einstieg in die Anwendung über eine definierte Startup-Logik, die in der `Program.cs` und den Konfigurationsdateien verankert ist.

6.3.1 Programm-Initialisierung (Program.cs)

Die `Program.cs` ist der "Entry Point" der ASP.NET Core Anwendung. Hier wird der "Generic Host" gebaut, der Dependency Injection Container befüllt und die HTTP-Request-Pipeline (Middleware) konfiguriert.

```
1    var builder = WebApplication.CreateBuilder(args);
2
3    // 1. Service Registrierung (DI)
4    builder.Services.AddApplicationServices(builder.Configuration);
5    builder.Services.AddControllers();
6    builder.Services.AddEndpointsApiExplorer();
7    builder.Services.AddSwaggerGen();
8
9    var app = builder.Build();
10
11   // 2. HTTP Request Pipeline
12   if (app.Environment.IsDevelopment())
13   {
14       app.UseSwagger();
15       app.UseSwaggerUI();
16   }
17
18   app.UseHttpsRedirection();
19   app.UseCors("AllowWebApp");
```

```

20 app.UseAuthentication();
21 app.UseAuthorization();
22 app.MapControllers();
23
24 app.Run();

```

Listing 6.1: Auszug aus der Middleware-Konfiguration

6.3.2 Konfiguration (appsettings.json)

Die Anwendungskonfiguration folgt dem *12-Factor App*-Prinzip: Alle umgebungsabhängigen Parameter werden aus der Datei `appsettings.json` geladen und können zur Laufzeit durch Umgebungsvariablen überschrieben werden (z.B. in Docker Compose). Das folgende Listing zeigt die vollständige Konfigurationsstruktur (sensible Werte wie Passwörter sind aus Sicherheitsgründen ausgeblendet):

```

1 {
2   "ConnectionStrings": {
3     "DefaultConnection": "Host=<db-host>;Port=5433;Database=EduShelfDb;..."
4   },
5   "AIService": {
6     "Provider": "Ollama",
7     "Endpoint": "http://<ollama-host>:11434",
8     "ChatModel": "qwen3:14b",
9     "EmbeddingModel": "nomic-embed-text",
10    "VisionModel": "moondream:1.8b",
11    "ContextLengthLimit": 4096,
12    "Prompts": {
13      "System": "...", // Steuert das Grundverhalten des LLMs
14      "Intent": "...", // Anleitung zur JSON-Intent-Erkennung
15      "Flashcard": "...", // Anleitung zur Flashcard-Generierung
16      "Quiz": "...", // Anleitung zur Quiz-Generierung
17    }
18  },
19  "Minio": {
20    "Endpoint": "<minio-host>:9000",
21    "AccessKey": "<key>",
22    "SecretKey": "<secret>",
23    "BucketName": "edushelf-documents",
24    "Secure": false
25  },
26  "EmailSettings": {
27    "SmtpHost": "<smtp-server>",
28    "SmtpPort": 587,
29    "SmtpUser": "<user>",
30    "SmtpPass": "<password>",
31    "FromEmail": "<sender>"
32  },
33  "Seeding": {
34    "AdminPassword": "<admin-pw>",
35    "StudentPassword": "<student-pw>"

```

```
36 |   }  
37 | }
```

Listing 6.2: Konfigurationsstruktur der appsettings.json

Die wichtigsten Konfigurationsblöcke im Überblick:

- **ConnectionStrings:** Verbindungsstring zur PostgreSQL-Datenbank (Host, Port, Datenbankname, Zugangsdaten).
- **AIService:** Zentrale KI-Konfiguration. `ChatModel` gibt das Sprachmodell an (z.B. `qwen3:14b`), `EmbeddingModel` das Modell für die Vektorerzeugung (`nomic-embed-text`) und `VisionModel` das Modell für die Bildanalyse. `ContextLengthLimit` (Standard: 4096 Token) begrenzt die Kontextfenstergröße. Die `Prompts`-Untersektion enthält die vollständigen System-Prompts für alle KI-gesteuerten Features als konfigurierbare Strings – so können diese ohne Codeänderung angepasst werden.
- **Minio:** Verbindungsdaten zum S3-kompatiblen Object Storage (Endpoint, Zugangsdaten, Bucket-Name).
- **EmailSettings:** SMTP-Konfiguration für den Versand von Registrierungs-Bestätigungsmails.
- **Seeding:** Passwörter für vordefinierten Admin- und Student-Benutzer, die beim ersten Start automatisch angelegt werden.

6.4 Datenmodelle und Datenbankdesign

Das Datenmodell wurde entworfen, um sowohl relationale Benutzerdaten als auch unstrukturierte Dokumenteninhalte effizient zu verwalten. EF Core Migrationen (`Migrations` Ordner) stellen sicher, dass das Datenbankschema synchron zum Code bleibt.

6.4.1 Kernelemente

- **User:** Die zentrale Identitätseinheit. Enthält `Username`, `Email`, `PasswordHash` (gespeichert als BCrypt-Hash), sowie Felder für die E-Mail-Verifizierung (`IsEmailConfirmed`, `EmailConfirmationToken`).

- **Role & UserRole:** Implementierung einer Many-to-Many Beziehung für ein rollenbasiertes Zugriffssystem (RBAC). Standardrollen sind z.B. SStudent oder Admin".
- **Document:** Repräsentiert Metadaten eines hochgeladenen Dokuments. Das Feld `Path` verweist auf den Speicherort im MinIO-Bucket. Es besteht eine 1:n-Beziehung zum User.
- **DocumentShare:** Ermöglicht das Teilen von Dokumenten zwischen Benutzern (Many-to-Many via Join-Table). Diese Entität steuert den Zugriff für Nicht-Besitzer.
- **Cascading Deletes:** Die Datenbankkonfiguration (`OnModelCreating`) stellt sicher, dass beim Löschen eines Nutzers alle abhängigen Daten (Dokumente, Chats, Favoriten) automatisch entfernt werden (`DeleteBehavior.Cascade`), um Datenwaisen zu vermeiden.
- **DocumentChunk:** Dies ist die wichtigste Entität für die semantische Suche. Ein Dokument wird in viele kleine Abschnitte zerlegt.
 - **Content:** Der rohe Textinhalt des Abschnitts.
 - **Embedding:** Ein Vektor vom Typ `vector(768)`, der die semantische Bedeutung des Textes repräsentiert.
 - **Page:** Referenz zur ursprünglichen Seite im Dokument (für Zitate).
- **ChatSession & ChatMessage:** Bilden den Verlauf einer Konversation ab. `ChatSession` gruppiert Nachrichten, während `ChatMessage` den eigentlichen Inhalt (User-Input oder AI-Response) speichert.

6.4.2 Entity Relationship Diagram (ERD)

Das folgende Diagramm visualisiert die Beziehungen zwischen den Entitäten. Zentral ist der **User**, der Dokumente besitzt und Lerninhalte (Flashcards, Quiz-zen) erstellt.

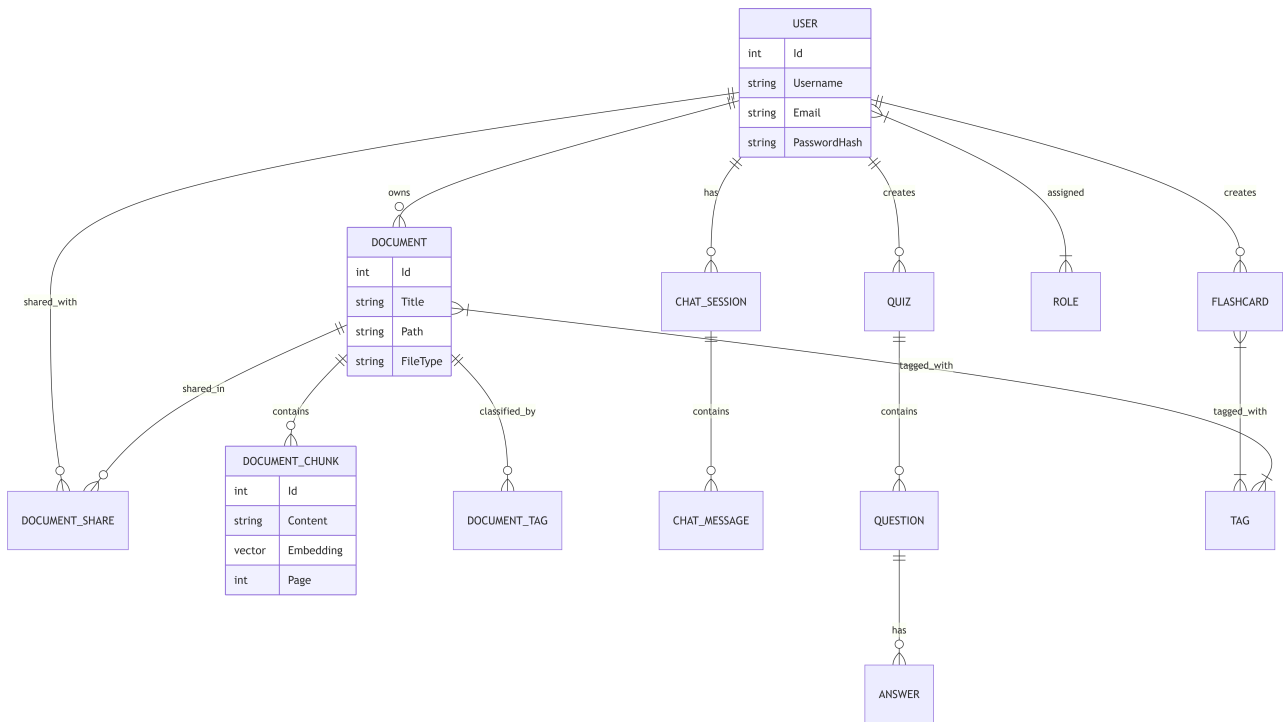


Abbildung 6.1: Entity Relationship Diagram des EduShelf Backends

6.5 API Design und Controller

Die Kommunikation mit dem Frontend erfolgt über eine RESTful API. Die acht Controller sind thematisch gruppiert und folgen REST-Konventionen (GET, POST, PUT, PATCH, DELETE). Nahezu alle Endpunkte sind hinter dem [Authorize]-Attribut geschützt, d.h. sie erfordern eine aktive Cookie-Session. Ausnahmen sind die öffentlichen Authentifizierungs-Endpunkte (Registrierung, Login, E-Mail-Bestätigung).

6.5.1 Ausgewählte API-Endpunkte

Methode	Route	Auth	Beschreibung
api/users			
POST	/	Anonym	Registrierung
POST	/login	Anonym	Login, setzt HttpOnly-Cookie
POST	/logout	Auth	Logout, löscht Session
GET	/me	Auth	Eigenes Profil abrufen
DELETE	/id	Admin	Nutzer löschen
api/documents			
GET	/	Auth	Eigene Dokumente (paginiert)
POST	/	Auth	Dokument hochladen (startet Indexierungs-Job)
DELETE	/id	Auth	Dokument inkl. MinIO-Datei löschen
GET	/download/id	Auth	Datei-Stream herunterladen
POST	/id/share	Auth	Dokument per E-Mail teilen
GET	/search	Auth	Volltext- und Tag-Suche
api/chat			
POST	/message	Auth	Nachricht senden, löst RAG-Pipeline aus
GET	/sessions	Auth	Chat-Sitzungen abrufen
POST	/sessions	Auth	Neue Chat-Sitzung erstellen
api/flashcards & api/quizzes			
POST	/flashcards/generate	Auth	Lernkarten per KI generieren
POST	/quizzes/generate	Auth	Quiz per KI generieren
api/tags			
GET	/	Auth	Alle Tags abrufen
POST	/	Admin	Neuen Tag erstellen

Tabelle 6.1: Wichtigste API-Endpunkte des EduShelf-Backends

6.6 Sicherheitskonzepte

Sicherheit war ein integraler Bestandteil des Designs („Security by Design“).

6.6.1 Authentifizierung & Autorisierung

Die Authentifizierung erfolgt über Cookies (`CookieAuthenticationDefaults`).

- **Login-Prozess:** Der Nutzer sendet Anmeldedaten an `POST /api/users/login`. Der `AuthService` lädt den Benutzer inklusive seiner Rollen aus der Datenbank und überprüft den Passwort-Hash mittels `BCrypt`. Zusätzlich wird geprüft, ob die E-Mail-Adresse bereits bestätigt wurde – nicht-verifizierte Accounts können sich nicht einloggen.
- **Claims-Based Identity:** Bei erfolgreichem Login werden Claims (kryptographisch signierte Aussagen) über den Benutzer in das Cookie eingebettet: Benutzer-ID (`ClaimTypes.NameIdentifier`), E-Mail, Benutzername und alle zugewiesenen Rollen (`ClaimTypes.Role`). Diese Claims stehen serverseitig bei jedem Request über `HttpContext.User` zur Verfügung.
- **Session-Lebensdauer:** Das Cookie ist auf 120 Minuten befristet (`IsPersistent = true`, `ExpiresUtc = +120 min`).
- **Middleware:** Die Standard-ASP.NET Core Authentifizierungsmiddleware entschlüsselt das Cookie bei jedem Request und rekonstruiert das `ClaimsPrincipal`-Objekt.

```
1 var claims = new List<Claim>
2 {
3     new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
4     new Claim(ClaimTypes.Email, user.Email),
5     new Claim(ClaimTypes.Name, user.Username)
6 };
7 foreach (var userRole in user.UserRoles)
8     claims.Add(new Claim(ClaimTypes.Role, userRole.Role.Name));
9
10 var claimsIdentity = new ClaimsIdentity(
11     claims, CookieAuthenticationDefaults.AuthenticationScheme);
12 var authProperties = new AuthenticationProperties
13 {
14     IsPersistent = true,
15     ExpiresUtc = DateTimeOffset.UtcNow.AddMinutes(120)
16 };
17 await _httpContextAccessor.HttpContext.SignInAsync(
18     CookieAuthenticationDefaults.AuthenticationScheme,
19     new ClaimsPrincipal(claimsIdentity),
20     authProperties);
```

Listing 6.3: Claims-Aufbau beim Login in `AuthService.cs`

6.6.2 Rollenbasiertes Zugriffssystem (RBAC)

EduShelf implementiert ein einfaches, aber effektives Role-Based Access Control System. Rollen (**Student**, **Admin**) werden in der Tabelle **Roles** gespeichert und über die Join-Tabelle **UserRoles** Nutzern zugewiesen.

Jeder neu registrierte Nutzer erhält automatisch die Rolle **Student**. Die Rolle wird beim Erstellen des Nutzers aus der Datenbank geladen und dem **UserRoles**-Set hinzugefügt:

```
1 var studentRole = await _context.Roles
2   .SingleOrDefaultAsync(r => r.Name == Roles.Student);
3 user.UserRoles.Add(new UserRole { Role = studentRole });
```

Listing 6.4: Automatische Rollenzuweisung bei der Registrierung

In den Controllern wird die Autorisierung deklarativ über Attribute gesteuert:

- **[Authorize]** – Nur eingeloggte Nutzer (jede Rolle).
- **[Authorize(Roles = "Admin")]** – Nur Administratoren (z.B. für Nutzer-Löschung oder Tag-Verwaltung).

6.6.3 E-Mail-Verifizierung

Um sicherzustellen, dass Nutzer nur mit gültigen E-Mail-Adressen registriert werden können, implementiert EduShelf einen vollständigen E-Mail-Bestätigungsflow.

6.6.3.1 Registrierungsvorgang

Bei der Registrierung wird zunächst auf doppelte Benutzernamen und E-Mails geprüft. Dann wird ein kryptographisch sicherer Bestätigungstoken generiert:

```
1 var user = new User
2 {
3     Username      = userRegister.Username,
4     Email         = userRegister.Email,
5     PasswordHash  = BCrypt.Net.BCrypt.HashPassword(userRegister.Password),
6     IsEmailConfirmed = false,
7     EmailConfirmationToken =
8         Convert.ToHexString(RandomNumberGenerator.GetBytes(32)),
9     EmailConfirmationTokenExpiresAt = DateTime.UtcNow.AddHours(24)
10 };
11 // ...
12 var confirmationLink =
13     $"{appUrl}/confirm-email?email={Uri.EscapeDataString(user.Email)}" +
14     "&token={Uri.EscapeDataString(user.EmailConfirmationToken)}";
```

```
15 await _emailService.SendEmailAsync(user.Email, "Confirm your email",  
    emailBody);
```

Listing 6.5: Token-Generierung und E-Mail-Versand in AuthService.cs

Der Token ist ein zufälliger 32-Byte-Wert (hexadezimal kodiert, 64 Zeichen lang), der 24 Stunden gültig ist. Schlägt der E-Mail-Versand fehl, wird der Nutzer-Datensatz sofort wieder aus der Datenbank gelöscht – d.h. das System ist *atomic*: Es existiert kein unverifizierter Account ohne zugehörige Bestätigungsmail.

6.6.3.2 Bestätigungsvorgang

Der Nutzer klickt den Link im Frontend, das ihn an `POST /api/users/confirm-email` weiterleitet. Der `AuthService` führt dabei vier Prüfungen durch:

1. Nutzer mit dieser E-Mail existiert.
2. E-Mail ist noch nicht bestätigt (Idempotenz – doppelte Klicks sind harmlos).
3. Der übermittelte Token stimmt exakt mit dem gespeicherten überein.
4. Der Token ist nicht abgelaufen (`ExpiresAt > DateTime.UtcNow`).

Bei Erfolg werden `IsEmailConfirmed = true` gesetzt und Token sowie Ablaufzeit aus der Datenbank gelöscht.

6.6.4 Datenvalidierung (Fluent Validation)

Eingehende Daten werden rigoros validiert. Die Bibliothek *FluentValidation* erlaubt die Definition von komplexen Regeln in dedizierten Validator-Klassen.

Beispiel `UserRegisterValidator`:

- **Username**: Pflichtfeld, 3–20 Zeichen.
- **Email**: Pflichtfeld, muss gültiges E-Mail-Format haben.
- **Password**: Pflichtfeld, mindestens 6 Zeichen.

Diese Validierung geschieht automatisch vor dem Controller-Aufruf. Ungültige Anfragen werden sofort mit HTTP 400 (Bad Request) und einer strukturierten Fehlerliste abgelehnt.

6.6.5 Error Handling Middleware

Ein globales `ErrorHandlingMiddleware` fängt alle unbehandelten Ausnahmen ab. Dies verhindert, dass interne Implementierungsdetails (Stack Traces) an den Client durchsickern. Stattdessen werden standardisierte JSON-Fehlermeldungen und passende HTTP-Statuscodes zurückgegeben:

Exception-Typ	HTTP-Statuscode
<code>NotFoundException</code>	404 Not Found
<code>BadRequestException</code>	400 Bad Request
<code>UnauthorizedAccessException</code>	401 Unauthorized
<code>ValidationException</code>	400 Bad Request
<code>Sonstige Exception</code>	500 Internal Server Error

Tabelle 6.2: Mapping von Exceptions auf HTTP-Statuscodes

6.7 Ingestion-Pipeline (Datenverarbeitung)

Damit Dokumente durchsuchbar werden, müssen sie eine komplexe Verarbeitungspipeline durchlaufen. Dieser Prozess wird vom `IndexingService` gesteuert und ist rechenintensiv.

6.7.1 1. Textextraktion (OCR & Parsing)

Der `FileParsingService` analysiert den Dateityp und wählt die passende Strategie:

- **PDF-Verarbeitung mit PdfPig:** Für PDF-Dateien wird die Bibliothek *UglyToad.PdfPig* verwendet. Sie erlaubt den Zugriff auf den rohen Textstream sowie die Extraktion von Bildern.
- **Hybride Bildanalyse:** Eingebettete Bilder werden isoliert und an ein lokales Vision-Model (via Ollama) gesendet. Der Prompt "Describe this image in detail" generiert eine textuelle Beschreibung, die in den Dokumentenindex einfließt. Dies ermöglicht die semantische Suche nach Bildinhalten (z.B. Diagramme, Graphen).
- **DOCX:** Nutzung des Open XML SDKs zum strukturierten Auslesen des `MainDocumentPart`.

6.7.2 2. Chunking (Intelligente Segmentierung)

LLMs haben ein begrenztes Kontextfenster, und die semantische Suche verliert bei zu großen Textblöcken an Präzision. Der **TextChunker** implementiert daher eine mehrstufige Strategie:

1. **Satz-Tokenisierung:** Der Text wird zunächst anhand von Satzzeichen (. ! ?) in einzelne Sätze zerlegt.
2. **Rekombination:** Diese Sätze werden iterativ zu einem Chunk zusammengefügt, bis eine konfigurierte Grenze von ca. 1024 Zeichen erreicht ist. Dies stellt sicher, dass semantische Zusammenhänge (Sätze) nicht willkürlich in der Mitte durchschnitten werden.
3. **Token-Limit:** Zusätzlich wird sichergestellt, dass die Chunks eine maximale Token-Anzahl (für das Embedding-Model) nicht überschreiten.

```

1 public static List<string> SplitBySentence(string text)
2 {
3     // Nutzung von Regex Lookbehind (?<=...), um das Satzzeichen zu behalten
4     var sentences = Regex.Split(text, @"(?<=[\.\!\?])\s+");
5     return new List<string>(sentences);
6 }

```

Listing 6.6: Implementierung der Satz-Segmentierung in TextChunker.cs

Das simple Splitten nach Zeichenanzahl würde Sätze zerreißen und den Kontext zerstören. Der hier gezeigte Ansatz garantiert semantische Integrität.

6.7.3 3. Embedding und Vektorspeicherung

Jeder Text-Chunk wird in einen hochdimensionalen Vektor umgewandelt und effizient gespeichert.

- **Embedding-Modell:** Nutzung von state-of-the-art Modellen wie *nomi-embed-text*, die semantische Ähnlichkeiten im Vektorraum abbilden.
- **HNSW Index:** In der PostgreSQL-Datenbank wird für die Spalte **Embedding** ein HNSW (Hierarchical Navigable Small World) Index angelegt.

```
modelBuilder.Entity<DocumentChunk>()
```

```
.HasIndex(dc => dc.Embedding)
.HasMethod("hns")
.HasOperators("vector_12_ops");
```

Dieser Index ermöglicht eine extrem schnelle Nearest-Neighbor-Suche (ANN) auch bei Millionen von Vektoren, da nicht die gesamte Tabelle gescannt werden muss.

6.8 Detaillierte Analyse ausgewählter Module

Um die technische Tiefe zu verdeutlichen, wird hier exemplarisch die Implementierung sicherheitskritischer und komplexer Logik analysiert.

6.8.1 Authentifizierung: Secure Cookies und Hashing

Die Sicherheit der Benutzerdaten hat oberste Priorität. Der AuthService implementiert daher bewährte Industriestandards.

6.8.1.1 Passwort-Hashing (BCrypt)

Passwörter werden niemals im Klartext gespeichert. Stattdessen wird der BCrypt-Algorithmus verwendet, der durch seinen konfigurierbaren "Work Factor"(Kostenfaktor) resistent gegen Brute-Force- und Rainbow-Table-Angriffe ist.

```
1 // user.PasswordHash kommt aus der DB, passwordDto.Password vom User
2 bool isValid = BCrypt.Net.BCrypt.Verify(passwordDto.Password, user.
  PasswordHash);
3 if (!isValid)
4 {
5     throw new UnauthorizedAccessException("Invalid credentials");
6 }
```

Listing 6.7: Verifizierung im AuthService

6.8.1.2 Cookie-Konstruktion

Nach erfolgreichem Login wird kein JWT im LocalStorage (Frontend) abgelegt (XSS-Gefahr), sondern ein **HttpOnly Cookie** gesetzt.

- **HttpOnly:** Das Cookie kann nicht per JavaScript ausgelesen werden.
- **Secure:** Wird nur über HTTPS übertragen.
- **SameSite=Strict:** Das Cookie wird nur an den eigenen Origin gesendet (Schutz vor CSRF).

6.8.2 Ingestion-Pipeline: HNSW Indexierung

Wie in Abschnitt 6 beschrieben, nutzt das System einen HNSW-Index. Die technische Herausforderung liegt hier in der Konfiguration der Operatorenklasse für `pgvector`. Code-seitig wird dies in der `OnModelCreating`-Methode des `ApiDbContext` gelöst, um sicherzustellen, dass PostgreSQL den Index korrekt für L2-Distanz-Berechnungen optimiert. Dies ist ein direktes Mapping von C#-Code auf datenbankspezifische Features (Code-First Migration").

6.9 RAG (Retrieval-Augmented Generation) Workflow

Das Herzstück von EduShelf ist die RAG-Pipeline: Anstatt das LLM allein auf sein allgemeines Wissen zu stützen, wird jede Benutzeranfrage zunächst dazu genutzt, relevante Textabschnitte aus den hochgeladenen Dokumenten abzurufen. Diese Abschnitte werden dann als *Kontext* in den Prompt eingefügt, bevor das LLM zur Antwort aufgefordert wird. Dies reduziert Halluzinationen und ermöglicht dokumentenspezifische, präzise Antworten.

6.9.1 Systemübersicht und Datenfluss

Das folgende Diagramm visualisiert den vollständigen Anfrage-Antwort-Zyklus im RAG-System und zeigt, wie die verschiedenen Services zusammenwirken.

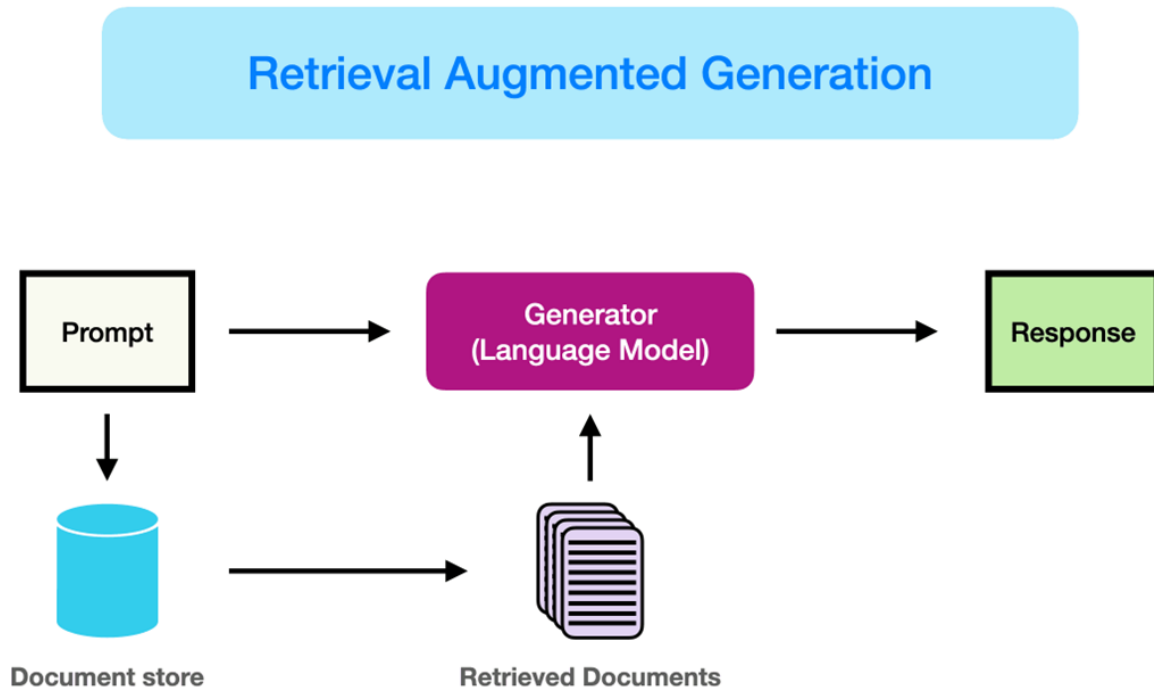


Abbildung 6.2: Architekturdiagramm des RAG-Systems in EduShelf

Der Datenfluss bei einer Benutzeranfrage lässt sich in fünf klar abgegrenzte Schritte gliedern:

1. **Anfrage & Intent-Erkennung:** Der `ChatController` empfängt die Nachricht des Nutzers und leitet sie an den `ChatService` weiter. Dieser übergibt sie zuerst an den `IntentDetectionService`.
2. **Vektorielle Suche (Retrieval):** Basierend auf dem erkannten Intent ruft der `RetrievalService` die semantisch relevantesten `DocumentChunks` aus der PostgreSQL/pgvector-Datenbank ab.
3. **Kontext-Aufbereitung:** Der `PromptGenerationService` kombiniert die Konversationshistorie, die abgerufenen Chunks und die aktuelle Frage zu einem strukturierten Prompt.
4. **LLM-Inferenz:** Über das Microsoft Semantic Kernel SDK wird der Prompt an das lokal laufende Ollama-LLM gesendet.

5. **Antwort & Persistenz:** Die generierte Antwort wird in der Datenbank als `ChatMessage` gespeichert und an den Client zurückgegeben.

6.9.2 Schritt 1: Intent-Erkennung (`IntentDetectionService`)

Bevor eine semantische Suche stattfindet, muss das System verstehen, *was* der Nutzer möchte. Der `IntentDetectionService` sendet die Benutzeranfrage an das LLM mit einem speziellen System-Prompt, der das Modell anweist, eine strukturierte JSON-Antwort zu liefern:

```
1 var chatHistory = new ChatHistory();
2 chatHistory.AddSystemMessage(
3     _configuration.GetValue<string>("AIService:Prompts:Intent")
4     ?? "Identify the intent.");
5 chatHistory.AddUserMessage(userInput);
6
7 var result = await chatCompletionService.GetChatMessageContentAsync(
8     chatHistory);
9 var cleanedJson = JsonHelper.ExtractJson(result.Content ?? string.Empty);
10 var intent = JsonSerializer.Deserialize<Intent>(cleanedJson,
11     new JsonSerializerOptions { PropertyNameCaseInsensitive = true })
12     ?? new Intent { Type = "question", DocumentName = null };
```

Listing 6.8: Intent-Erkennung via LLM in `IntentDetectionService.cs`

Das `Intent`-Objekt enthält zwei Felder:

- **Type:** Klassifiziert die Anfrage als "question", "summarize", "flashcards" oder "quiz".
- **DocumentName:** Falls der Nutzer explizit ein Dokument nennt (z.B. „Erkläre mir Kapitel 3 aus Physik.pdf“), wird der Dateiname hier extrahiert.

Dieser vorgelagerte Schritt ermöglicht eine fundamentale Weichenstellung im weiteren Ablauf: Statt immer dieselbe RAG-Pipeline auszuführen, wird die Antwort-Strategie dynamisch angepasst.

6.9.3 Schritt 2: Semantische Suche (`RetrievalService`)

Der `RetrievalService` ist für die Beschaffung der relevanten Dokumentenabschnitte zuständig. Er implementiert dabei eine zweistufige Strategie:

6.9.3.1 Primäre Suche: Name-basiertes Matching

Falls der Intent einen `DocumentName` enthält, sucht der Service direkt nach Chunks, deren übergeordnetes Dokument diesen Namen trägt. Dies erfolgt über einen PostgreSQL-spezifischen, groß-/kleinschreibungsunabhängigen Vergleich (`ILike`):

```

1 var chunks = await _context.DocumentChunks
2   .Include(dc => dc.Document)
3   .Where(dc => dc.Document.UserId == userId &&
4     (EF.Functions.ILike(dc.Document.Title, intent.DocumentName) ||
5     EF.Functions.ILike(dc.Document.Title,
6     documentNameWithoutExtension)))
7   .ToListAsync();

```

Listing 6.9: Dokumenten-spezifische Suche in `RetrievalService.cs`

6.9.3.2 Fallback: Semantische Vektorsuche

Wenn kein Dokumentname angegeben wurde oder keine Treffer gefunden wurden, wird eine semantische Suche über alle Chunks des Nutzers durchgeführt. Die Anfrage wird in einen Embedding-Vektor umgewandelt und mit allen gespeicherten Chunk-Vektoren per L2-Distanz verglichen:

```

1 var promptEmbedding = await _embeddingGenerator.GenerateAsync(userInput);
2 var k = GetDynamicK(userInput); // 5, 10 oder 20 Chunks
3
4 return await _context.DocumentChunks
5   .Include(dc => dc.Document)
6   .Where(dc => dc.Document.UserId == userId)
7   .OrderBy(dc => dc.Embedding.L2Distance(new Vector(promptEmbedding.Vector)))
8   .Take(k)
9   .ToListAsync();

```

Listing 6.10: Embedding-basierte Vektorsuche in `RetrievalService.cs`

Der Wert k (Anzahl der zurückgegebenen Chunks) ist dynamisch: Bei kurzen Anfragen (unter 10 Wörtern) werden $k = 5$, bei mittleren $k = 10$ und bei komplexen Anfragen (über 30 Wörter) $k = 20$ Chunks abgerufen.

6.9.4 Schritt 3: Prompt-Aufbau (PromptGenerationService)

Der PromptGenerationService baut eine vollständige ChatHistory auf, die drei Elemente enthält:

- Eine **System-Instruktion**, die das Verhalten des LLMs grundlegend steuert.
- Die **bisherige Konversationshistorie** der aktuellen Chat-Sitzung (für kontextsensitive Folgefragen).
- Die **aktuelle Benutzeranfrage**, angereichert mit dem abgerufenen Dokumentenkontext.

```

1 // Kontext aus Chunks zusammenstellen und auf Token-Limit kuerzen
2 foreach (var chunk in relevantChunks) {
3     contextText.AppendLine($"[{chunk.Document.Title}] {chunk.Content}");
4 }
5 contextString = _tokenService.Truncate(contextString, contextLengthLimit);
6
7 // Finale Benutzernachricht mit eingebettetem Kontext
8 finalUserMessage.AppendLine($"[Context from Documents]:\n{contextString}\n")
9 ;
10 finalUserMessage.AppendLine($"[User Question]: {userInput}");
11 chatHistory.AddUserMessage(finalUserMessage.ToString());

```

Listing 6.11: Prompt-Konstruktion in PromptGenerationService.cs

Ein zentrales Element ist die **Token-Limitierung** durch den ITokenService. Um das Kontextfenster des LLMs (konfigurierbar, Standard: 4096 Token) nicht zu überschreiten, werden die Chunks vor der Übergabe tokenisiert und präzise abgeschnitten (*Smart Truncation*). Dies verhindert API-Fehler und stellt sicher, dass die semantisch nächsten Chunks priorisiert werden.

6.9.5 Schritt 4: Orchestrierung und Intent-basiertes Branching (ChatService)

Der ChatService fungiert als zentraler Orchestrator und trifft nach der Retrieval-Phase eine **Intent-basierte Fallunterscheidung**. Dies unterscheidet EduShelf von einem einfachen Chatbot:

```

1 var intent = await _intentDetectionService.GetIntentAsync(promptInput);
2 var relevantChunks = await _retrievalService.GetRelevantChunksAsync(
3     promptInput, userId, intent);
4

```

```

5 if (intent.Type == "flashcards") {
6     await _flashcardService.GenerateFlashcardsAsync(
7         new GenerateFlashcardsRequest { Context = context, Count = 5 },
8         userId);
9     responseContent = "I have generated flashcards.";
10 }
11 else if (intent.Type == "quiz") {
12     await _quizService.GenerateQuizAsync(
13         new GenerateQuizRequest { Context = context, Count = 5 }, userId);
14     responseContent = "I have generated a quiz.";
15 }
16 else { // "question" oder "summarize"
17     var chatHistory = _promptGenerationService.BuildChatHistory(
18         chatSession, relevantChunks, promptInput);
19     var result = await chatCompletionService.GetChatMessageContentAsync(
20         chatHistory);
21     responseContent = result.Content ?? "Sorry, no response generated.";
22 }

```

Listing 6.12: Intent-basiertes Branching in ChatService.cs

Dieses Design ermöglicht es, über eine einzige Chat-Schnittstelle drei fundamentale Lernfunktionen anzubieten: konversationelles Frage-Antwort, automatische Flashcard-Generierung und automatische Quiz-Erstellung – alles gesteuert durch die Intention hinter der natürlichsprachlichen Eingabe des Nutzers.

6.9.6 Zusammenfassung: Retrieval-Augmented Generation

Die Implementierung des RAG-Systems stellt sicher, dass das LLM stets auf den spezifischen Kontext der hochgeladenen Dokumente des Nutzers antwortet. Die wichtigsten Designprinzipien sind:

- **Datenprivacy:** Durch lokale Ollama-Inferenz verlassen keine Dokumenteninhalte den Server.
- **Präzision:** Durch semantische Vektorsuche und optionales Name-Matching werden nur die relevantesten Chunks an das LLM übergeben.
- **Robustheit:** Token-basierte Kontextlimitierung verhindert LLM-Abstürze durch überlange Prompts.
- **Erweiterbarkeit:** Das Intent-Branching erlaubt es, neue KI-gestützte Features durch Hinzufügen weiterer Intent-Typen zu integrieren, ohne die Kernpipeline zu ändern.

6.10 KI-gestützte Lerninhaltsgenerierung

Neben der RAG-basierten Chat-Funktionalität bietet EduShelf zwei weitere KI-Features: die automatische Generierung von **Lernkarten (Flashcards)** und **Multiple-Choice-Quizzes**. Beide Features nutzen denselben Ollama-LLM-Stack, folgen aber einem eigenständigen, direkteren Prompt-Workflow, da hier keine Vektorsuche benötigt wird – der Nutzer liefert den Kontext direkt über Dokumenten-IDs oder RAG-abgerufene Chunks.

6.10.1 Generierungspipeline (5 Schritte)

Beide Services (`FlashcardService`, `QuizService`) folgen derselben fünfstufigen Pipeline:

1. **Kontextbeschaffung:** Der Service akzeptiert drei Eingabequellen: einen direkten `Context-String` (aus der RAG-Pipeline), eine einzelne `DocumentId` oder eine Liste von `DocumentIds`. Im letzteren Fall werden die Textinhalte aller Dokumente via `GetDocumentContentAsync` abgerufen und mit einem Trennzeichen (`\n\n__NEXT_DOCUMENT__\n\n`) zusammengefügt.
2. **Kontext-Truncation:** Um das Kontextfenster des LLMs nicht zu sprengen, wird der Dokumenteninhalt auf maximal 12.000 Zeichen (Flashcards) bzw. 15.000 Zeichen (Quizze) begrenzt.
3. **Prompt-Vorbereitung:** Der System-Prompt wird aus `appsettings.json` geladen (`AIService.Prompts.Flashcard` bzw. `:Quiz`). Die Variable `{Count}` wird durch die gewünschte Anzahl ersetzt.
4. **LLM-Inferenz:** Das Modell wird mit dem vorbereiteten `ChatHistory`-Objekt aufgerufen.
5. **JSON-Parsing und Persistenz:** Die Antwort wird geparkt und in der Datenbank gespeichert.

6.10.2 Flashcard-Generierung (FlashcardService)

Der LLM wird angewiesen, eine JSON-Liste von Objekten mit den Feldern **Question** und **Answer** zurückzugeben. Das folgende Snippet zeigt den Prompt-Aufbau und die anschließende Verarbeitung:

```

1 // Prompt zusammenbauen und KI aufrufen
2 var chatHistory = new ChatHistory();
3 chatHistory.AddSystemMessage(
4     promptTemplate.Replace("{Count}", request.Count.ToString()));
5 chatHistory.AddUserMessage(
6     $"Text to analyze:\n\n{documentContent}\n\n" +
7     "IMPORTANT: Output strictly valid JSON. Start with [."");
8
9 var result = await chatCompletionService
10     .GetChatMessageContentAsync(chatHistory);
11
12 // JSON bereinigen und deserialisieren
13 var cleanedJson = JsonHelper.ExtractJson(result.Content);
14 var flashcards = JsonSerializer.Deserialize<List<GeneratedFlashcardJson>>(
15     cleanedJson,
16     new JsonSerializerOptions { PropertyNameCaseInsensitive = true });

```

Listing 6.13: KI-Aufruf und JSON-Parsing in FlashcardService.cs

Nach dem Parsing werden die generierten Karten automatisch mit einem Tag versehen, der den Dokumenttitel trägt, und als **Flashcard**-Entitäten in der Datenbank persistiert.

6.10.3 Quiz-Generierung (QuizService)

Die Quiz-Generierung ist komplexer, da das LLM ein verschachteltes JSON-Objekt mit **Title**, **Questions** und pro Frage mehrere **Answers** (jeweils mit **IsCorrect**-Flag) produzieren muss. Da LLMs trotz strikter Anweisung gelegentlich vom erwarteten Schema abweichen (z.B. alternative Feldnamen wie **options** statt **Answers**), implementiert der **QuizService** eine **zweistufige Parsing-Strategie**:

```

1 var jsonNode = JsonNode.Parse(cleanedJson);
2
3 // Stufe 1: Wrapper-Handling (falls LLM { "quiz": {...} } liefert)
4 if (jsonNode["quiz"] != null)
5     jsonNode = jsonNode["quiz"];
6
7 // Stufe 2a: Standard-Deserialisierung versuchen
8 var result = jsonNode.Deserialize<GeneratedQuizJson>(options);
9
10 // Stufe 2b: Fallback -- Manuelles Parsing bei alternativen Schemas
11 if (result == null || !result.Questions.Any())
12 {

```

```
13 // Felder wie "text"/"Text", "options"/"answers", "answer" (string)
14 // werden manuell ausgelesen und normalisiert.
15 result = ManuallyParseQuizJson(jsonNode);
16 }
```

Listing 6.14: Robustes JSON-Parsing in QuizService.cs

Die manuelle Fallback-Stufe erkennt dabei sowohl Antwortoptions als einfache Strings (`[Paris", London", ...]`) als auch als Objekte (`[{text: ..., isCorrect: ...}]`). Dies macht die Quiz-Generierung robust gegenüber variierenden LLM-Antwortformaten.

Nach erfolgreichem Parsing wird das Quiz als `Quiz`-Entität mit allen `Question`- und `Answer`-Objekten in einer einzigen EF Core-Transaktion gespeichert.

6.11 Infrastruktur & Deployment

Das System ist vollständig containerisiert ("Dockerized"), um eine konsistente Umgebung von der Entwicklung bis zur Produktion zu gewährleisten.

6.11.1 Docker-Komposition

Die `docker-compose.yml` definiert das Zusammenspiel der Dienste und deren spezifische Konfiguration für die Produktionsumgebung.

6.11.1.1 Datenbank (PostgreSQL mit pgvector)

Für die persistente Speicherung und Vektorsuche wird ein spezialisiertes Image verwendet:

- **Image:** `pgvector/pgvector:pg16` - Dies ist essentiell, da das Standard-Postgres-Image die `vector`-Erweiterung nicht enthält.
- **Ports:** Das Mapping `5433:5432` vermeidet Konflikte mit lokalen Postgres-Instanzen auf dem Host.
- **Healthcheck:** Ein integrierter Healthcheck (`pg_isready`) stellt sicher, dass abhängige Container (wie die API) erst starten, wenn die Datenbank vollständig hochgefahren ist.

- **Persistenz:** Das Volume `postgres_data` sichert die relationalen Daten und Vektor-Indizes über Container-Neustarts hinweg.

6.11.1.2 KI-Inferenz (Ollama)

Der Ollama-Container ist für die lokale Ausführung der LLMs zuständig und benötigt direkten Hardware-Zugriff für performante Inferenz:

- **GPU-Passthrough:** Über die `deploy.resources`-Sektion wird der Nvidia-Treiber durchgereicht:

```
reservations:
  devices:
    - driver: nvidia
      count: 1
      capabilities: [gpu]
```

Dies ermöglicht die Nutzung von CUDA-Kernen, was die Antwortzeit (Time-to-First-Token) drastisch reduziert (von Sekunden auf Millisekunden).

- **Persistenz:** Das Volume `ollama_data` speichert die heruntergeladenen Modelle (z.B. Llama 3) im `/root/.ollama` Verzeichnis, um erneute Downloads bei Container-Updates zu vermeiden.
- **Keep-Alive:** Die Umgebungsvariable `OLLAMA_KEEP_ALIVE` verhindert, dass das Modell nach kurzer Inaktivität aus dem VRAM entladen wird, was die Latenz bei aufeinanderfolgenden Anfragen minimiert.

6.11.1.3 Object Storage (MinIO)

MinIO dient als skalierbarer, S3-kompatibler Dateispeicher.

- **Ports:**
 - 9000: API-Port für die Kommunikation mit dem Backend (S3-Protokoll).
 - 9001: Web-Interface (Console) zur manuellen Verwaltung von Buckets und Dateien.

- **Command:** Der Startbefehl `server /data -console-address :9001` aktiviert das Web-UI explizit.
- **Volumes:** `minio_data` speichert die physischen Dateien (PDFs, Bilder) persistent.

6.11.2 Dockerfile

Das `Dockerfile` der API nutzt Multi-Stage Builds, um das Image klein zu halten:

1. **Build Stage:** Nutzt das vollständige SDK Image (`mcr.microsoft.com/dotnet/sdk:8.0`), um den Code zu kompilieren und Abhängigkeiten (NuGet Pakete) zu laden.
2. **Runtime Stage:** Nutzt das schlanke Runtime Image (`mcr.microsoft.com/dotnet/aspnet:8.0`), welches nur die zum Ausführen nötigen Komponenten enthält. Das kompilierte Artefakt aus Stage 1 wird hierhin kopiert.

Dies reduziert die Image-Größe drastisch und verbessert die Sicherheit, da keine Compiler-Tools im Produktions-Container vorhanden sind.

6.12 Resilienz und Stabilität

In verteilten Systemen, insbesondere bei der Anbindung von KI-Diensten, sind temporäre Ausfälle unvermeidbar. Um die Robustheit der Anwendung zu gewährleisten, wurde die Bibliothek `Polly` integriert.

6.12.1 Retry-Policies & Circuit Breaker

In der `ServiceCollectionExtensions.cs` werden resiliente HTTP-Clients konfiguriert:

1. **Retry Policy:** Bei transienten Fehlern (z.B. Netzwerk-Glitch, 503 Service Unavailable) wird der Request bis zu 3-mal wiederholt, mit einem exponentiellen Backoff (2^n Sekunden).
2. **Circuit Breaker:** Wenn 5 Fehler in Folge auftreten, "öffnet" sich der Stromkreis für 30 Sekunden. Alle weiteren Anfragen werden sofort abgeblockt, um das überlastete Subsystem (z.B. Ollama) zu schonen.


```

1 var retryPolicy = HttpPolicyExtensions
2   .HandleTransientHttpError()
3   .WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2,
4     retryAttempt)));
5 var circuitBreakerPolicy = HttpPolicyExtensions
6   .HandleTransientHttpError()
7   .CircuitBreakerAsync(5, TimeSpan.FromSeconds(30));

```

Listing 6.15: Polly-Konfiguration

6.13 Herausforderungen und Lösungen

Während der Backend-Entwicklung mussten mehrere technische Hürden genommen werden.

6.13.1 Blockierende Operationen bei großen Dateien

Lösung: Einführung einer `BackgroundJobQueue`. Der Upload-Endpoint nimmt die Datei entgegen, speichert sie in MinIO und reiht nur eine "Job-ID" in die Queue ein, bevor er sofort 202 Accepted zurückgibt. Ein Hintergrunddienst ("Worker") arbeitet diese Queue asynchron ab.

```

1 protected override async Task ExecuteAsync(CancellationToken stoppingToken)
2 {
3     while (!stoppingToken.IsCancellationRequested)
4     {
5         var workItem = await _queue.DequeueAsync(stoppingToken); //
6         // Blockiert bis Arbeit da ist
7         try
8         {
9             await workItem(stoppingToken); // Fuehrt den eigentlichen
10            // Indexing-Task aus
11        }
12        catch (Exception ex)
13        {
14            _logger.LogError(ex, "Error occurred executing background work
15            item.");
16        }
17    }
18 }

```

Listing 6.16: Verarbeitungsschleife im BackgroundJobService

6.13.2 LLM Kontext-Fenster-Limits

Problem: GPT-4 und andere Modelle haben ein festes Limit an Tokens (z.B. 4096 oder 8192). Ein blindes Einfügen von Dokumenten würde zu Abstürzen führen. **Lösung:** Implementierung des `TokenService` mit *Tiktoken*. Bevor ein Prompt generiert wird, werden die Tokens der abgerufenen Chunks gezählt. Ist das Limit erreicht, werden die am wenigsten relevanten Chunks verworfen (SSmart Truncation").

6.13.3 Datenkonsistenz bei Löschoperationen

Problem: Wenn ein User gelöscht wird, dürfen dessen Dokumente und Vektoren nicht als "Datenmüllin der Datenbank verbleiben. **Lösung:** Nutzung von `OnDelete(DeleteBehavior.Cascade)` in EF Core. Da auch die Vektoren (`DocumentChunk`) über Foreign Keys hart an das `Document` und dieses an den `User` gekoppelt ist, löscht ein einziger SQL-Befehl (`DELETE FROM Users WHERE Id=...`) kaskadierend den gesamten assoziierten Datenbaum.

6.14 Qualitätssicherung und Tests

Die Stabilität des Backends wird durch eine umfassende automatisierte Test-Suite auf Basis von `xUnit` gewährleistet. Das Testprojekt `EduShelf.Api.Tests` enthält 16 Testklassen, die Geschäftslogik, Controller, Validatoren und Hilfsfunktionen abdecken.

Testklasse	Getestete Komponente
AuthServiceTests	Registrierung, Login, E-Mail-Bestätigung
FlashcardServiceTests	KI-Flashcard-Generierung, JSON-Fehler
QuizServiceTests	KI-Quiz-Generierung, Parsing-Robustheit
ChatServiceTests	RAG-Orchestrierung, Intent-Branching
DocumentServiceTests	Upload, Löschung, Indexierung
IntentDetectionServiceTests	JSON-Intent-Parsing
TokenServiceTests	Token-Limitierung und Truncation
ResilienceTests	Polly Retry- und Circuit-Breaker-Policies
UserRegisterValidatorTests	FluentValidation-Regeln
JsonHelperTests	JSON-Extraktion aus LLM-Antworten
DocumentsControllerTests	HTTP-Routing und Autorisierung
FlashcardsControllerTests	HTTP-Routing und Autorisierung
UsersControllerTests	HTTP-Routing und Autorisierung
TagsControllerTests	HTTP-Routing und RBAC
QuizCreateDtoValidatorTests	Eingabevalidierung für Quiz-DTOs

Tabelle 6.3: Übersicht der Test-Klassen im Testprojekt

6.14.1 Unit Tests (xUnit + Moq)

Die Geschäftslogik (Services) wird isoliert getestet. Externe Abhängigkeiten wie Datenbank, LLM oder E-Mail-Dienst werden mittels Moq durch kontrollierte Testdoubles ersetzt. Die Datenbank wird für jeden Testfall durch eine separate In-Memory-Datenbankinstanz (über `UseInMemoryDatabase` mit `Guid.NewGuid()`) vollständig isoliert.

6.14.1.1 Beispiel: FlashcardService

Die drei Kerntests des `FlashcardService` illustrieren das typische Testmuster:

```

1 [Fact]
2 public async Task
3     GenerateFlashcardsAsync_ValidContent_ShouldCreateFlashcards()
4 {
5     // Arrange -- Mock-LLM gibt valides JSON zurueck
6     var jsonResponse = "[{\"Question\": \"What is AI?\", \"Answer\": \"
7         Artificial Intelligence\"},\" +
8         \" {\"Question\": \"Is it cool?\", \"Answer\": \"Yes
9         \"}]\";
10
11     _mockChatCompletionService
12         .Setup(c => c.GetChatMessageContentsAsync(...))
13         .ReturnsAsync(new List<ChatMessageContent> { new(AuthorRole.
14             Assistant, jsonResponse) });
15
16     // Act
17     var result = await service.GenerateFlashcardsAsync(request, userId);

```

```

14 // Assert
15 Assert.Equal(2, result.ToList().Count);
16 Assert.Equal(2, context.Flashcards.Count());
17 Assert.Single(context.Tags); // Tag "Test Doc" automatisch erstellt
18 }

```

Listing 6.17: Unit Test: Erfolgreiche Flashcard-Generierung

```

1 [Fact]
2 public async Task GenerateFlashcardsAsync_NoContent_ShouldThrowBadRequest()
3 {
4     _mockDocumentService.Setup(d => d.GetDocumentContentAsync(1, userId,
5         Roles.Student))
6         .ReturnsAsync(""); // Leerer Inhalt
7     await Assert.ThrowsAsync<BadRequestException>(
8         () => service.GenerateFlashcardsAsync(request, userId));
9 }

```

Listing 6.18: Unit Test: Leerer Dokumenteninhalte wirft BadRequestException

```

1 [Fact]
2 public async Task
3     GenerateFlashcardsAsync_InvalidJsonFromAI_ShouldThrowKernelServiceException()
4 {
5     // Mock-LLM gibt Text statt JSON zurueck
6     var badResponse = new ChatMessageContent(AuthorRole.Assistant, "This is
7         not JSON");
8     _mockChatCompletionService.Setup(...).ReturnsAsync(new List<
9         ChatMessageContent> { badResponse });
10
11     await Assert.ThrowsAsync<KernelServiceException>(
12         () => service.GenerateFlashcardsAsync(request, userId));
13 }

```

Listing 6.19: Unit Test: Ungültiges JSON vom LLM wirft KernelServiceException

6.14.1.2 Beispiel: AuthService

Der AuthService wird ebenfalls umfassend getestet, inklusive des E-Mail-Bestätigungsflows:

```

1 [Fact]
2 public async Task RegisterUserAsync_ValidData_ShouldCreateUserAndSendEmail()
3 {
4     var result = await service.RegisterUserAsync(registerDto);
5
6     Assert.Single(context.Users);
7     Assert.False((await context.Users.FirstAsync()).IsEmailConfirmed);
8     Assert.NotNull((await context.Users.FirstAsync()).EmailConfirmationToken);
9
10    // Verifiziert: E-Mail wurde genau einmal an die richtige Adresse
    // gesendet

```

```

11     _mockEmailService.Verify(
12         x => x.SendEmailAsync("new@example.com", It.IsAny<string>(), It.
13             IsAny<string>()),
14         Times.Once);

```

Listing 6.20: Unit Test: Registrierung erstellt Nutzer und sendet E-Mail

```

1 [Fact]
2 public async Task ConfirmEmailAsync_ExpiredToken_ShouldThrowBadRequest()
3 {
4     var user = new User
5     {
6         EmailConfirmationToken = "expired_token",
7         EmailConfirmationTokenExpiresAt = DateTime.UtcNow.AddHours(-1) // in
8             der Vergangenheit
9     };
10    context.Users.Add(user);
11
12    await Assert.ThrowsAsync<BadRequestException>(
13        () => service.ConfirmEmailAsync(confirmDto));

```

Listing 6.21: Unit Test: Abgelaufener Token wirft BadRequestException

6.14.2 Validatoren-Tests

FluentValidation-Regeln werden mit der Bibliotheksmethode `TestValidate` isoliert geprüft – kein HTTP-Stack nötig:

```

1 [Fact]
2 public void Should_Have_Error_When_Email_Is_Invalid()
3 {
4     var model = new UserRegister { Username = "user", Email = "invalid_email",
5         Password = "pw" };
6     var result = _validator.TestValidate(model);
7     result.ShouldHaveValidationErrorFor(x => x.Email);
8 }
9
10 [Fact]
11 public void Should_Not_Have_Error_When_Model_Is_Valid()
12 {
13     var model = new UserRegister { Username = "validUser", Email = "test@example.com",
14         Password = "strongPassword123" };
15     _validator.TestValidate(model).ShouldNotHaveAnyValidationErrors();

```

Listing 6.22: Validator-Tests mit FluentValidation.TestHelper

6.14.3 Integration Tests

Für Controller und die Datenbankinteraktion werden Integrationstests verwendet. Hierbei wird mittels `WebApplicationFactory` ein echter Test-Server im Speicher hochgefahren, der gegen den vollständigen ASP.NET Core Middleware-Stack (Routing, Authentifizierung, Fehlerbehandlung) und eine separate In-Memory-Datenbankinstanz läuft. Dies stellt sicher, dass das Routing, die Autorisierungsattribute und das Entity Framework korrekt zusammenspielen – unabhängig voneinander, aber als zusammenhängendes System.

6.15 Fazit

Das Backend von EduShelf stellt ein robustes, modernes und erweiterungsfähiges Fundament für die Lernplattform dar. Die während der Entwicklung getroffenen Architekturentscheidungen haben sich als tauglich erwiesen, um die komplexen Anforderungen einer KI-gestützten Dokumentenverwaltungsplattform zu erfüllen.

6.15.1 Erreichte Ziele

- **Vollständige RAG-Pipeline:** Die Kombination aus semantischer Vektorsuche (pgvector + HNSW-Index), Intent-Erkennung und tokenbasierter Kontextlimitierung ermöglicht präzise, dokumentenspezifische KI-Antworten ohne externe API-Abhängigkeiten.
- **Datenschutz durch lokale KI:** Der Einsatz von Ollama als lokaler LLM-Inferenzserver stellt sicher, dass keine Dokumenteninhalte das eigene System verlassen. Dies ist insbesondere für Bildungseinrichtungen relevant, die mit sensiblen Unterrichtsmaterialien arbeiten.
- **Robuste Lerninhaltsgenerierung:** Sowohl die Flashcard- als auch die Quiz-Generierung sind durch mehrstufige JSON-Parsing-Strategien widerstandsfähig gegenüber variierenden LLM-Antwortformaten.
- **Sicherheit:** Der vollständige E-Mail-Verifizierungsflow, BCrypt-Passwort-Hashing, HttpOnly-Cookie-Authentifizierung und deklaratives RBAC schaffen eine mehrschichtige Sicherheitsarchitektur.

- **Infrastruktur:** Die vollständige Containerisierung mit Docker Compose erlaubt ein reproduzierbares, portables Deployment auf jedem Linux-Server – ohne manuelles Setup.
- **Testabdeckung:** 16 Testklassen mit Unit- und Integrationstests für alle kritischen Services, Controller und Validatoren stellen die langfristige Wartbarkeit sicher.

6.15.2 Lernerkenntnisse und Herausforderungen

Die größte technische Herausforderung war die **Unvorhersehbarkeit von LLM-Ausgaben**: Trotz präziser System-Prompts und expliziter JSON-Formatvorgaben weichen Sprachmodelle gelegentlich vom erwarteten Schema ab. Die Lösung lag nicht im Erzwingen einer bestimmten Ausgabe, sondern in der Implementierung mehrerer Fallback-Strategien im Parsing-Layer – eine Erkenntnis, die fundamental für jede produktive LLM-Integration ist.

Ein weiteres Lernfeld war die **Performance-Optimierung der Ingestion-Pipeline**: Das initiale Ansatz, Dokumente synchron im HTTP-Request-Zyklus zu verarbeiten, führte bei großen PDFs zu Frontend-Timeouts. Die Einführung einer **BackgroundJobQueue** mit asynchroner Verarbeitung löste dieses Problem elegant, ohne die Architektur grundlegend zu verändern.

6.15.3 Mögliche Erweiterungen

- **Streaming-Responses:** Die aktuelle Implementierung wartet auf die vollständige LLM-Antwort. Eine Server-Sent-Events (SSE) oder WebSocket-Anbindung würde die wahrgenommene Latenz deutlich reduzieren.
- **Multi-Tenant-Unterstützung:** Aktuell ist jeder User vollständig isoliert (keine gemeinsamen Wissensbasen). Eine optionale Kurs- oder Gruppenebene wäre eine natürliche Erweiterung für den Bildungskontext.
- **Mikroservice-Migration:** Der monolithische Ansatz war für die Initialentwicklung ideal. Bei steigender Last könnten rechenintensive Services (Indexierung, KI-Inferenz) als eigenständige Mikroservices ausgelagert werden.

Zeiterfassung



Abbildung 6.3: AI und iOS Zeitaufwand

Gesamt: 263,17h

Freizeit: 228,48h

Abbildungsverzeichnis

4.1	Konzeptionelle Entwicklung der Idee – von der Problemerkennung zur Lösung	27
5.1	Hauptdashboard von EduShelf im Dark-Mode	30
5.2	Registrierungsformular von EduShelf im Dark-Mode	33
5.3	Login-Ansicht von EduShelf im Dark-Mode	35
5.4	Navigationsleiste von EduShelf mit rollenbasierter Anzeige (Dark-Mode)	36
5.5	Dateiansicht von EduShelf (Dark-Mode)	38
5.6	AI und iOS Zeitaufwand	53
6.1	Entity Relationship Diagram des EduShelf Backends	62
6.2	Architekturdiagramm des RAG-Systems in EduShelf	71
6.3	AI und iOS Zeitaufwand	88

Code-Verweise

5.1	Paralleles Datenladen mit Promise.all im Dashboard	30
5.2	Filterung von Shared Files anhand des localStorage-Status	31
5.3	Clientseitige Eingabevalidierung im Registrierungsformular	33
5.4	Verarbeitung von Backend-Validierungsfehlern	34
5.5	Outlet-Pattern in MainLayout.jsx	36
5.6	Rollenbasierte Anzeige des Admin-Menüpunkts	36
5.7	Dynamische CSS-Klasse bei aktivem Navigationselement	37
5.8	Formularlogik im ShareFileModal	38
5.9	Persistierung der Share-Entscheidungen im localStorage	39
5.10	Auszug aus der Vite-Konfiguration	41
5.11	Routing-Struktur in App.jsx	42
5.12	Generische GET-Methode im API-Service	43
5.13	CSS für den Flip-Effekt	45
5.14	Theme-Switching mittels CSS-Variablen	47
5.15	Initialer State im QuizModal	48
5.16	Update-Logik für verschachtelte Antworten	48
5.17	Nutzung eines Sets fuer UI-State	49
5.18	Asynchrones Laden von Detaildaten	50
5.19	Dockerfile für das Frontend	51
6.1	Auszug aus der Middleware-Konfiguration	58
6.2	Konfigurationsstruktur der appsettings.json	59
6.3	Claims-Aufbau beim Login in AuthService.cs	64
6.4	Automatische Rollenzuweisung bei der Registrierung	65
6.5	Token-Generierung und E-Mail-Versand in AuthService.cs	65
6.6	Implementierung der Satz-Segmentierung in TextChunker.cs . . .	68
6.7	Verifizierung im AuthService	69
6.8	Intent-Erkennung via LLM in IntentDetectionService.cs	72
6.9	Dokumenten-spezifische Suche in RetrievalService.cs	73
6.10	Embedding-basierte Vektorsuche in RetrievalService.cs	73
6.11	Prompt-Konstruktion in PromptGenerationService.cs	74

6.12	Intent-basiertes Branching in ChatService.cs	74
6.13	KI-Aufruf und JSON-Parsing in FlashcardService.cs	77
6.14	Robustes JSON-Parsing in QuizService.cs	77
6.15	Polly-Konfiguration	81
6.16	Verarbeitungsschleife im BackgroundJobService	81
6.17	Unit Test: Erfolgreiche Flashcard-Generierung	83
6.18	Unit Test: Leerer Dokumenteninhalt wirft BadRequestException	84
6.19	Unit Test: Ungültiges JSON vom LLM wirft KernelServiceEx- ception	84
6.20	Unit Test: Registrierung erstellt Nutzer und sendet E-Mail . . .	84
6.21	Unit Test: Abgelaufener Token wirft BadRequestException . . .	85
6.22	Validator-Tests mit FluentValidation.TestHelper	85